解説資料:AI 海難救助システム

実装編(プログラムの中身の解説)

Physical AI 応用講座1 最終課題の付録 ／ omnicampus アカウント: ba2vwbus ／ 2026 年 10 月 ／ コード: GitHub ba2vwbus13/pai-ai-sea-rescue-board

この資料について

「レポート編」(1〜6 章)の続きである。ここでは、プログラムの中身を説明する。章の番号はレポート編から続けて、7 章から始める。

式やコードから入らずに、**具体的な数字の例と図**を先に見せてから、最後に「コードではどこか」を示す。各章の終わりの「設計上の考慮点」では、部品が止まった場合の動作や、設定を変えた場合の影響などをまとめた。行番号は GitHub に置いたコードのものである。

目次

7. システム構成と ROS 2
8. PX4 の飛行制御
9. YOLO による人物検出
10. 画像座標から地上座標への変換
11. mission の状態遷移
12. LoRa 通信の模擬
13. フォイルボードの制御
14. 評価と動画作成

章は、プログラムどうしのつなぎ方(7・8 章)、目にあたる AI(9・10 章)、頭にあたる mission(11 章)、救助側の無線とボード(12・13 章)、評価(14 章)の順に並べた。データが流れる順番である。

7 章 システム構成と ROS 2

7.1 ROS 2 の仕組み

このシステムのプログラムの多くは、ROS 2 の上で動いている。ROS 2 は、**いくつものプログラムを同時に動かし、プログラム同士でデータを受け渡しさせる仕組み**である。ROS 2 の上で動く 1 つのプログラムを、**ノード**という。たとえば、写真から人を見つける detector、ドローンを飛ばす mission、表示画面の dashboard は、どれもノードである。

ノードは、データを渡す相手を直接呼び出さない。データは、名前を付けた**トピック**に流す。トピックの名前は、`/ai_patrol/detections` のように、/ で区切って付ける。データを流すことを publish、トピックを聞いてデータを受け取ることを subscribe という。トピックに 1 回で流すデータのまとまりを、**メッセージ**という。メッセージにどんな項目が入るかは、**メッセージの型**として前もって決めてある。送る側と聞く側が同じ型を使うので、聞く側は項目を正しく読み取れる。

たとえば、detector は人を見つけると、見つけた人の北と東の位置、確信度、写真を受け取った時刻をメッセージに入れて、トピック `/ai_patrol/detections` に流す。このトピックは、mission と dashboard の 2 つのノードが聞いている。従って、detector が 1 回流すだけで、同じメッセージが 2 つのノードに届く(図 7-1)。

トピックは、ラジオのチャンネルにたとえられる。送る側は「どのチャンネルに流すか」だけを決め、聞く側は「どのチャンネルを聞くか」だけを決める。このようにすることで、送る側と聞く側はお互いを知らなくてよいので、ノードを 1 つ止めたり、入れ替えたりしやすい。たとえば、ボードの動き方を変えたいときは board だけを直せばよく、detector や mission は変えなくてよい。

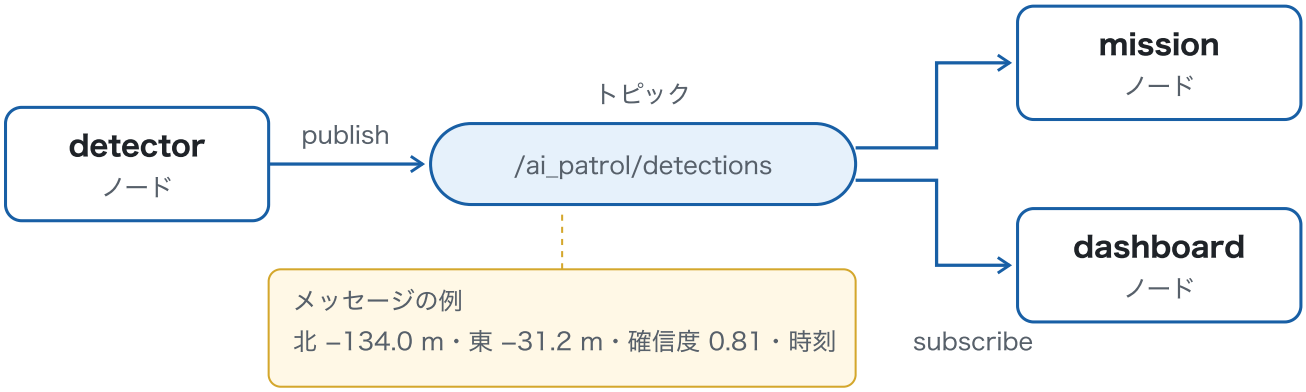


図 7-1 トピックによるメッセージの受け渡し

7.2 プログラム間の通信方式

3 種類の通信

このシステムのプログラムは、Gazebo、PX4、今回作った ROS 2 のノードの 3 つのグループに分かれる。Gazebo は海・ドローン・ボード・遭難者を動かすシミュレータで、PX4 はドローンの飛行を制御するソフトである。3 つのグループは別々の開発者が作ったものなので、データの送り方も別々である。従って、このシステムでは、**3 種類の通信**を使い分けている。それぞれの通信が何と何をつなぎ、何に使われるかを、表 7-1 にまとめる。

図 7-2 は、3 種類の通信でつながったプログラムの全体図である。青い実線が ROS 2(DDS)、紫の点線が Micro XRCE-DDS、橙の実線が gz-transport である。青い点線の枠の中が ROS 2 のノードであり、枠の外の Gazebo

と PX4 は、橙と紫の線でつながる。PX4 は ROS 2 のノードではないので、MicroXRCEAgent が PX4 の代わりに ROS 2 のトピックを流したり聞いたりする。

図の矢印をカメラの画像から順にたどると、データの流れがわかる。Gazebo のカメラの画像は、橙の線で image_bridge に渡り、青い線で detector に届く。detector が見つけた遭難者の位置は mission に届き、mission は人の座標を lora_link を通して board に送る。board はボードの動きを計算し、橙の線で Gazebo の中のボードを動かす。一方、mission が決めた飛ぶ先は、青い線で MicroXRCEAgent に渡り、紫の線で PX4 に届く。PX4 はモーターの回転数を計算し、橙の線で Gazebo のドローンを動かす。

ROS 2 の通信には、**DDS**(Data Distribution Service)という規格を使う。ノードは起動すると、同じネットワークにいる相手を自動で探し、同じトピック名・同じ型のノードどうしを直接つなぐ。7.1 節で説明したトピックの受け渡しは、この仕組みで実現している。

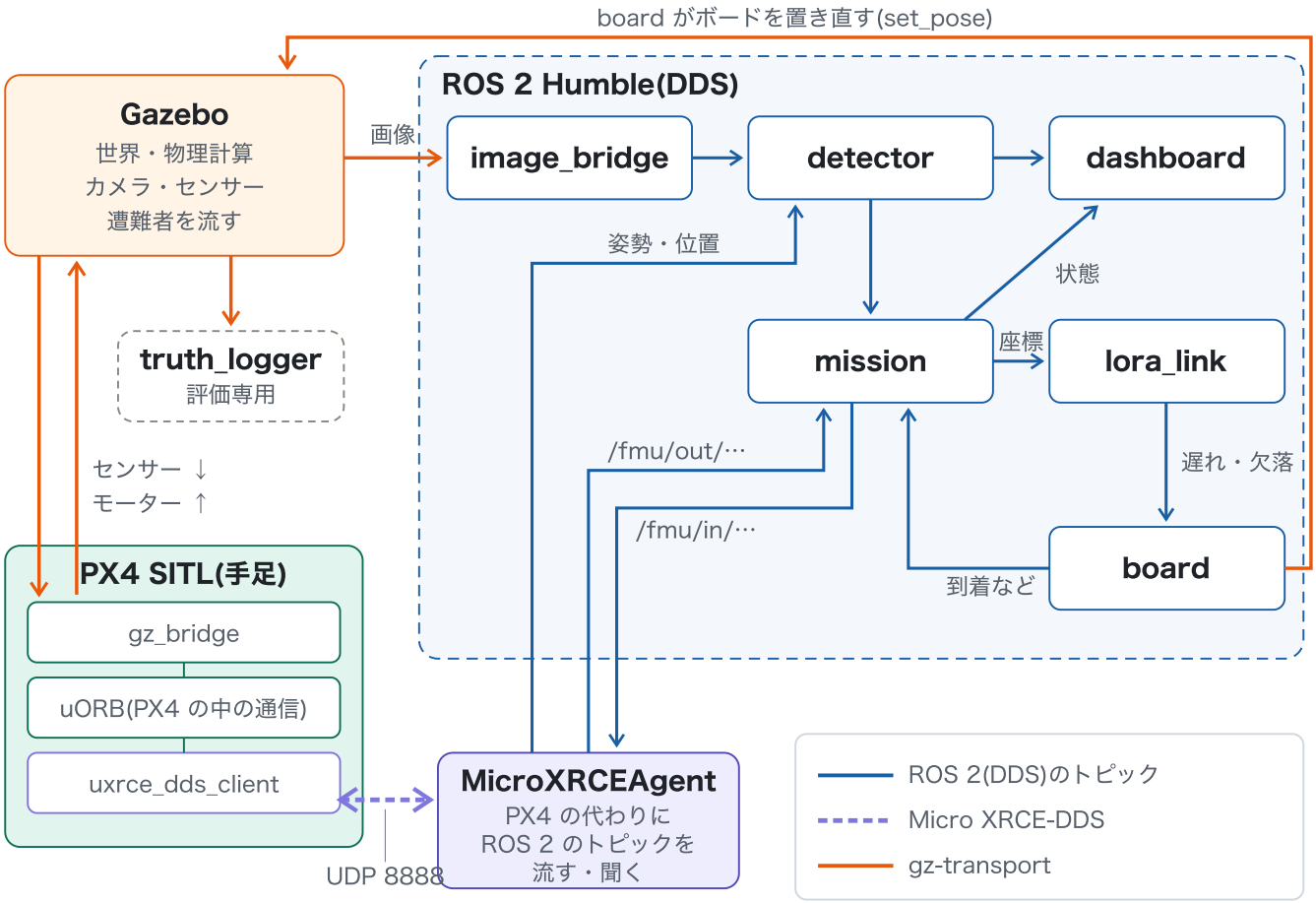


図 7-2 プログラム同士の通信の全体図

表 7-1 3 種類の通信

通信	何と何をつなぐか	このシステムでの例
ROS 2(DDS)	ROS 2 のノード同士。同じ PC の中でも、別の PC の間でも使える	detector → mission の検出結果、mission → lora_link → board の座標、board → mission のボードの状態
Micro XRCE-DDS	PX4 と ROS 2。PX4 の中の通信(uORB)を、ROS 2 のトピックに出し入れする	mission → PX4 の仮の目標点、PX4 → detector・mission の位置と姿勢(8.1 節)
gz-transport	Gazebo と、その外のプログラム。Gazebo 専用の通信	PX4 の gz_bridge が受け取るセンサーの値と、送るモーターの回転数。カメラ画像(image_bridge が ROS 2 に流し直す)。board がボードを置き直す。truth_logger が正解位置を聞く

Gazebo と ROS 2 の間の受け渡し

Gazebo の中のデータは gz-transport で流れ、ROS 2 のノードの間のデータは DDS で流れる。従って、Gazebo と ROS 2 の間では、両方の通信を使えるプログラムが、片方で受け取ったデータをもう片方で送り出す。図 7-3 では、箱の左右に付けた色付きの四角で、そのプログラムが使う通信を表した。橙は gz-transport、青は ROS 2、紫は Micro XRCE-DDS である。

この受け渡しをするのは、image_bridge と board である。image_bridge は、Gazebo のカメラ画像を ROS 2 のトピックに流し直す。board は、ROS 2 で受け取った遭難者の座標からボードの動きを計算し、Gazebo の中のボードを動かす。PX4 は、Gazebo とはセンサーの値とモーターの回転数をやりとりし、ROS 2 とは MicroXRCEAgent を通してやりとりする。PX4 と Gazebo の間は ROS 2 を通らないので、実機では、ここが本物のセンサーとモーターに置き換わる。truth_logger は、Gazebo から正解位置を受け取り、評価用のファイルに書くだけである。これら 4 つのプログラムが使う通信と、受け渡すものを、表 7-2 にまとめる。

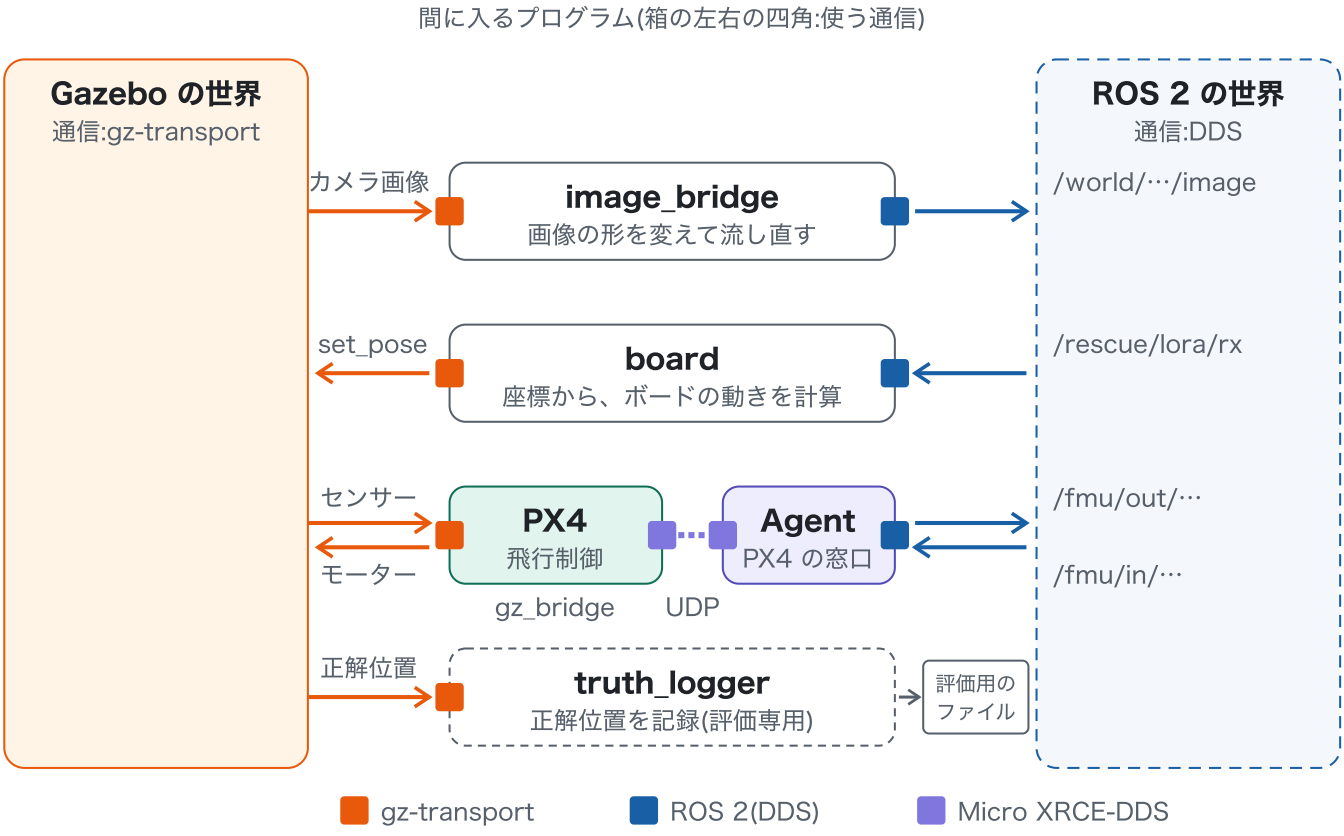


図 7-3 Gazebo と ROS 2 の間の受け渡し

表 7-2 Gazebo と ROS 2 の間に入るプログラム

プログラム	Gazebo との通信	ROS 2 との通信	受け渡すもの
image_bridge	gz-transport	DDS	カメラ画像を、Gazebo から ROS 2 へ。変換だけをする既製のノード(ros_gz パッケージ)
board	gz-transport	DDS	ROS 2 で受け取った座標から計算したボードの位置を、Gazebo へ。1 つのプログラムの中で 2 つの通信のライブラリを使う
PX4	gz-transport(gz_bridge)	Micro XRCE-DDS → MicroXRCEAgent → DDS	Gazebo とはセンサーの値とモーターの回転数。ROS 2 とは機体の位置・姿勢と、飛行の指令
truth_logger	gz-transport	(使わない。記録はファイルへ)	遭難者とボードの正解位置を、評価用のファイル truth_*.jsonl へ

7.3 トピック一覧

このシステムで使っているトピックを、表 7-3 に一覧にする。この表は、各ノードのコードにある `create_publisher` と `create_subscription` から集めたものである。

`in` は「PX4 へ入る」、`out` は「PX4 から出る」という意味である。PX4 のバージョンによって名前の後ろに `_v1` などが付くので、コードでは両方の名前を聞いている。

String(JSON) は、「文字列 1 本」だけを持つ一番簡単なメッセージの型に、JSON という書き方で複数の値を詰めたものである。自分たちで新しいメッセージの型を作らずに済むので、こうした。

表 7-3 海難救助システムで使っているトピック

トピック	メッセージの型	流す	聞く	中身
`/world/sea_board/…/camera/image`	Image	image_bridge	detector	カメラ画像 (1280 × 960)
`/fmu/out/vehicle_attitude`	VehicleAttitude	PX4	detector	機体の傾きと向き
`/fmu/out/vehicle_local_position`	VehicleLocalPosition	PX4	detector, mission	機体の位置(北・東・下)
`/fmu/out/vehicle_status`	VehicleStatus	PX4	mission	モード・アームしているか
`/ai_patrol/detections`	String(JSON)	detector	mission, dashboard	見つけた人の確信度・海の上の位置・写真を受け取った時刻
`/ai_patrol/annotated_image`	Image	detector	dashboard	枠を描いた画像(表示用)
`/fmu/in/vehicle_command`	VehicleCommand	mission	PX4	モード切り替え・アーム・着陸のお願い
`/fmu/in/offboard_control_mode`	OffboardControlMode	mission	PX4	「外から操縦するよ」の合図
`/fmu/in/trajectory_setpoint`	TrajectorySetpoint	mission	PX4	仮の目標点(位置・速さ・機首の向き)
`/ai_patrol/mission_state`	String(JSON)	mission	dashboard	今の状態・候補・出来事
`/rescue/lora/tx`	String(JSON)	mission	lora_link	ボードへ送る座標(ボードの発進・位置の更新)
`/rescue/lora/rx`	String(JSON)	lora_link	board	遅れて届いた座標(捨てられたものは来ない)
`/rescue/lora/stats`	String(JSON)	lora_link	mission	送信数・受信数・遅れ・乱数の種
`/rescue/boards`	String(JSON)	board	mission, dashboard	ボード 3 台の位置・速さ・状態(到着したか)
`/ai_patrol/dashboard`	Image	dashboard	(録画用)	表示画面の画像

7.4 ノードの基本構造

どのノードも、同じ形で書かれている。detector を例にする(`detector_node.py`)。

```
class DetectorNode(Node):
    def __init__(self):                          # ① 起動したときに 1 回だけ実行
        ...
        self.create_subscription(Image, 画像のトピック, self._on_image, ...)  # 聞く
        self.pub_det = self.create_publisher(String, '/ai_patrol/detections', 10)  # 流す準備
        self.create_timer(1.0, self._on_timer)  # 1 秒ごとに _on_timer を呼んでもらう

    def _on_image(self, msg):                    # ② 画像が届くたびに呼ばれる
        self.latest = (msg, 姿勢, 位置, time.time())   # 一番新しい画像を、その瞬間の姿勢・位置・時刻と取っておく

    def _on_timer(self):                         # ③ 1 秒ごとに呼ばれる
        msg, q, pos, t_img = self.latest
        ...YOLO で推論して、位置を計算...
        payload = dict(stamp=t_img, ...)         #    時刻は「写真を受け取った時刻」
        self.pub_det.publish(String(data=json.dumps(payload)))   # 流す

rclpy.spin(node)                                 # ④ 届くのを待ち続ける(止めるまで終わらない)
```

- **`create_subscription(型, トピック, 関数, …)`**:「このトピックに届いたら、この関数を呼んで」と登録する。自分で呼ぶのではなく、ROS 2 が呼ぶ。
- **`create_publisher(型, トピック, …)`**:流す準備。実際に流すのは `publish` を呼んだときである。
- **`create_timer(秒, 関数)`**:決まった間隔で関数を呼んでもらう。detector は 1 秒ごと、mission は 0.1 秒ごと、board は 0.05 秒ごとである。
- **QoS**(`BEST_EFFORT` など):届け方の約束。PX4 のトピックは「届かなくても送り直さない、最新の 1 つだけ」という約束で流れているので、聞く側も同じ約束にそろえる。

画像が届く(1 秒に何回も)ことと、推論する(1 秒に 1 回)ことを分けているのがポイントである。画像が届くたびに推論すると間に合わないので、届いたときは「取っておく」だけにしている。取っておくときに、**その瞬間の機体の姿勢・位置・時刻も一緒に**保存する。

そのため、計算で出てくるのは**「写真を受け取った瞬間に人がいた場所」**と、その時刻の組である。推論が終わるのは、写真を受け取ってから最大で 1 秒あまりあとになる。それでよい理由は次のとおりである。

- 写真に写っているのは、受け取った瞬間の人である。だから、**同じ瞬間**の機体の姿勢・位置と組み合わせないと、正しい場所にならない。
- もし推論が終わったときの機体の位置を使うと、捜索中は 1 秒で 7 m 進んでいるので、遭難者の位置も約 7 m ずれる。
- 人がその後に流れた分は、mission が予測位置の式(レポート編 3.5 節)で、時刻に合わせて足す。そのために、時刻も写真を受け取った瞬間のもの(`t_img`、136 行目)を付けている。

この時刻の付け方は、録画し直す前に直した。以前は、時刻に推論が**終わった**時刻(`time.time()`)を付けていた。すると、mission は遅れた分(最大 1 秒あまり)だけ、流れた量を少なく見積もる。海流 0.25 m/秒なら最大 0.25 m あまりのずれである。録画し直す前に、写真を受け取った時刻を付けるように直した(102 行目で保存、136 行目で使う)。

7.5 パラメータ

高度や確信度のしきい値など、あとで変えたくなる数字は、コードに直接書かずに**パラメータ**にしている。値は `config/params.yaml` に書く。

```
ai_detector:                    ← ノードの名前
  ros__parameters:
    conf_threshold: 0.30        # YOLO の確信度しきい値
    tile_alt_threshold: 12.0    # これより高いと 3x3 分割、低いと 2x2 分割
lora_link:
  ros__parameters:
    loss_rate: 0.03             # パケットロス率
    seed: -1                    # 乱数の種。負なら実行のたびに変える(使った種は統計に記録)
```

コードの側では、まず `declare_parameter('conf_threshold', 0.30)` で「この名前のパラメータを使う。書いてなければ 0.30」と宣言する。そのあと値を読み出す。detector では、読み出しを短く書くために、次の 1 行を使っている。

```
p = lambda n: self.get_parameter(n).value      # p('hfov') と書けば、hfov の値が出てくる
```

`lambda` は「名前のない小さな関数」を作る書き方である。`self.get_parameter('hfov').value` と毎回書く代わりに、`p('hfov')` と書けるようにしている。mission と board は、パラメータを `dict(…)` にまとめて宣言し、`self.cfg` や `self.p` に入れて使っている。

7.6 launch による一括起動

ノードを 1 つずつ起動するのは手間なので、`launch/ai_patrol.launch.py` で 6 つをまとめて起動する。`start_ai.sh` がこれを呼ぶ。`start_ai.sh` は、その前に評価用の truth_logger も起動する。

launch で起動するノードと、launch がそれぞれのノードに渡しているものを、表 7-4 に示す。

表 7-4 launch で起動するノード

ノード	渡しているもの
image_bridge	カメラのトピック名(Gazebo の画像を ROS 2 に流す。既製品)
detector	params.yaml、画像のトピック名、AI のモデルのファイル(GPU 用と CPU 用)
mission	params.yaml、記録の保存先
board	params.yaml。PYTHONPATH から AI 用の仮想環境を外して起動する。Gazebo の部品(gz-transport)と、AI 用の部品の protobuf の版がぶつかるためである
lora_link	params.yaml
dashboard	記録の保存先、全画面にするか、画面の位置

7.7 設計上の考慮点

lora_link が止まった場合の動作

lora_link が止まると、トピック `/rescue/lora/rx` に何も流れなくなる。従って、ボードは発進の合図を受け取れず、砂浜で待ったままになる。ただし、mission と detector は動き続けるので、ドローンは見守りを続ける。

画像の受け取りと推論を分ける理由

画像は 1 秒に何回も届くが、推論は 1 回に約 0.2 秒かかり、AI 以外の処理も合わせると 1 秒に 1 回が限度である。従って、`_on_image` の中では推論せず、届いた画像を取っておくだけにした。推論は、1 秒ごとに、その時点で一番新しい画像に対してだけ行う。

detector が付ける時刻

detector は、検出結果に、写真を受け取った時刻を付ける。もし推論が終わった時刻を付けると、人がその位置にいた時刻が、実際より遅く記録される。mission は「その時刻から今まで」に流れた分を足すので、足す分が少なくなり、予測位置が人の後ろ、つまり上流側へずれる。

8 章 PX4 の飛行制御

8.1 Micro XRCE-DDS による PX4 と ROS 2 の接続

PX4 は ROS 2 のノードではない。PX4 の中のプログラムどうしは、**uORB** という PX4 専用の通信でデータを受け渡している。たとえば、ドローンの位置を推定するプログラムは、推定したドローンの位置を uORB の `vehicle_local_position` に流し、飛行制御のプログラムがそれを聞く。

この uORB のデータを ROS 2 のトピックに出し、ROS 2 からの指令を uORB に入れるのが **Micro XRCE-DDS** である。XRCE は eXtremely Resource Constrained Environments(資源がとても限られた環境)の略である。

クライアントとエージェントの分担

本物のドローンでは、PX4 はフライトコントローラという小さなマイコンの上で動く。メモリは数百 KB〜数 MB しかない。DDS は、相手を自動で探す仕組みや、届け方の細かい約束を持つ大きな仕組みなので、マイコンには重すぎる。

そこで、仕事を 2 つに分ける。

- **クライアント**(`uxrce_dds_client`):PX4 の中で動く小さなプログラム。uORB のデータを詰めて送り、届いたデータを uORB に流すだけをする。
- **エージェント**(`MicroXRCEAgent`):PC(機上コンピュータ)で動くプログラム。PX4 の**代わりに**、ROS 2 のトピックを流したり聞いたりする。
  - PX4 から届いた位置や姿勢を、 `/fmu/out/vehicle_local_position_v1` などのトピックとして**流す**(publish)。
  - mission が流した `/fmu/in/trajectory_setpoint` などのトピックを**聞き**(subscribe)、PX4 へ渡す。

  ROS 2 のトピックを流したり聞いたりできるのは、DDS に参加しているプログラムだけである。PX4 は DDS に参加できないので、エージェントが PX4 の窓口として参加する。mission から見ると、エージェントが流すトピックは PX4 自身が流しているのと区別がつかない。

図 8-1 は、データが PX4 から mission まで、どの区間をどの方法で運ばれるかを示したものである。PX4 の中では、uORB で運ばれる。PX4 とエージェントの間は UDP で運ばれ、実機ではシリアルか Ethernet に変わる。エージェントから先は、ROS 2(DDS)で運ばれる。

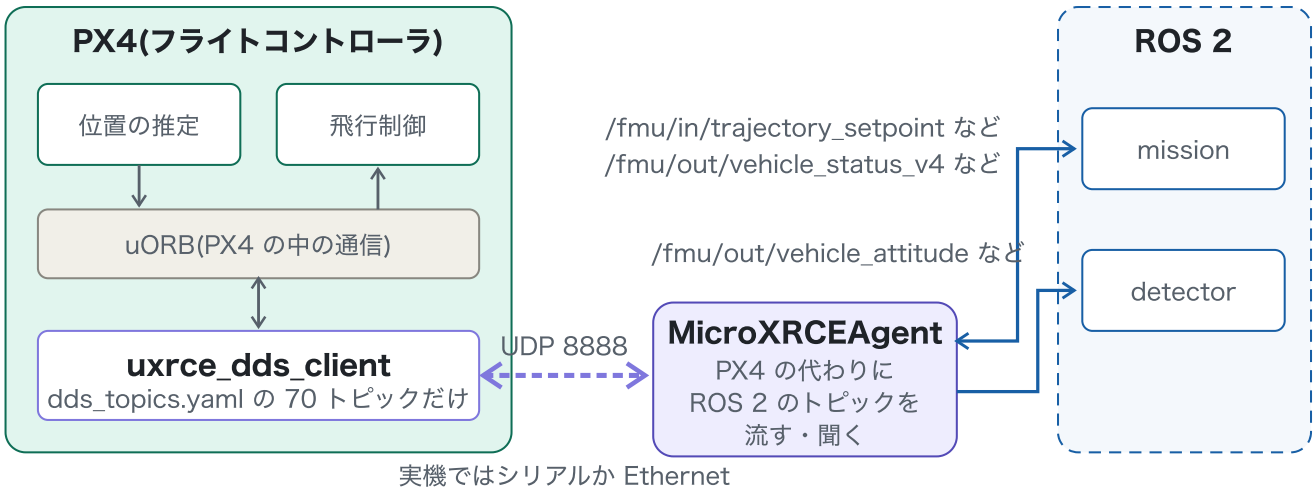


図 8-1 Micro XRCE-DDS の仕組み

メッセージの経路

1 つのメッセージが、どの順番で受け渡されるかを、表 8-1 に示す。左の列は、ドローンの位置が PX4 から mission に届くまでである。右の列は、仮の目標点が mission から PX4 に届くまでである。

表 8-1 ドローンの位置と仮の目標点がたどる道

#	PX4 → mission(位置)	mission → PX4(仮の目標点)
1	位置の推定が、uORB の `vehicle_local_position` に流す	mission が `TrajectorySetpoint` を作り、`/fmu/in/trajectory_setpoint` に publish する
2	uxrce_dds_client が uORB から読み、1 秒に 50 回までに間引いて送る	エージェントが DDS から受け取り、UDP でクライアントへ送る
3	UDP の 8888 番ポートを通ってエージェントへ届く	クライアントが受け取り、uORB の `trajectory_setpoint` に流す
4	エージェントが `/fmu/out/vehicle_local_position_v1` として DDS に publish する	PX4 の Offboard モードのプログラムが uORB から読み、飛行制御に渡す
5	mission の `_on_pos` が呼ばれる	機体が仮の目標点へ向かって動く

設定

このシステムで、Micro XRCE-DDS をどう設定しているかを、表 8-2 にまとめる。

表 8-2 Micro XRCE-DDS まわりの設定

項目	中身
クライアントの起動	PX4 の起動スクリプト(`ROMFS/px4fmu_common/init.d-posix/rcS`)の `uxrce_dds_client start -t udp -p 8888` 。PX4 を起動すると自動で動く
エージェントの起動	`start_ai.sh` の `MicroXRCEAgent udp4 -p 8888` 。まだ動いていなければ起動する。止めるのは `stop_all.sh`
どのトピックを出し入れするか	PX4 の `src/modules/uxrce_dds_client/dds_topics.yaml` に書いてある 70 トピックだけ。PX4 の中の uORB のトピックは数百あるが、全部は出さない
送る頻度の上限	同じファイルの `rate_limit` 。ドローンの姿勢と位置は 1 秒に 50 回、状態(vehicle_status)は 5 回まで。mission は 0.1 秒ごとに動くので、ドローンの位置は十分に新しい
メッセージの型	ROS 2 側は `px4_msgs` パッケージの型を使う。中身は PX4 の `msg/*.msg` と同じで、**PX4 と同じ版のものをビルドしておく必要がある**。版がずれると、項目の並びが合わずに正しく読めない
名前の後ろの `_v1` ・ `_v4`	新しい PX4 では、型を変えたメッセージに版の番号が付く。この PX4 では VehicleStatus が 4 版、VehicleLocalPosition が 1 版なので、トピック名は `/fmu/out/vehicle_status_v4` 、 `/fmu/out/vehicle_local_position_v1` になる。版 0 の型(VehicleAttitude、VehicleCommand、TrajectorySetpoint など)には付かない。コードでは、版が付く名前と付かない名前の両方を聞いている
届け方の約束(QoS)	PX4 側は「送り直さない(BEST_EFFORT)・最新の 1 つだけ(KEEP_LAST 1)」で流す。聞く側の約束が「必ず届ける(RELIABLE)」だと、DDS は相性が合わないとしてつながない。そのため、mission と detector の QoS をそろえている(7.4 節)
時刻	PX4 のメッセージの `timestamp` は、マイクロ秒の整数である。クライアントには、エージェントと時刻を合わせる仕組みがある。mission は ROS 2 の時計からマイクロ秒を作って入れている(`_now_us` 、219 行目)

座標系 NED と ENU

位置は、3 つの数で表す。PX4 と Gazebo では、この 3 つの数の並べ方と、高さの向きがちがう。PX4 は「北・東・下」の順に並べ、下をプラスに数える。この数え方を **NED** という。Gazebo は「東・北・上」の順に並べ、上をプラスに数える。この数え方を **ENU** という。

たとえば、ドローンが離陸地点から南へ 140 m・西へ 47 m の所の上空、高度 18 m にいるとする。北と東をプラスとするので、この位置は北 −140 m・東 −47 m である。PX4 はこの位置を (−140, −47, −18) と書き、Gazebo は (−47, −140, 18) と書く。同じ場所なのに、1 番目と 2 番目の数が入れ替わり、高さの符号が逆になる(図 8-2)。

Micro XRCE-DDS は、数をそのまま運ぶだけで、並べ替えない。従って、このシステムでは、mission と detector は PX4 と同じ NED のまま計算し、Gazebo とやりとりする board と truth_logger だけが、NED と ENU を変換している(13.5 節、14.1 節)。このようにすることで、変換を書く場所は 2 つのプログラムだけで済む。ただし、変換を忘れると北と東が入れ替わるので、ボードは北 −47 m・東 −140 m という、まったく別の場所に置かれてしまう。高さの符号は、8.3 節で PX4 に高度を送るときにも出てくる。

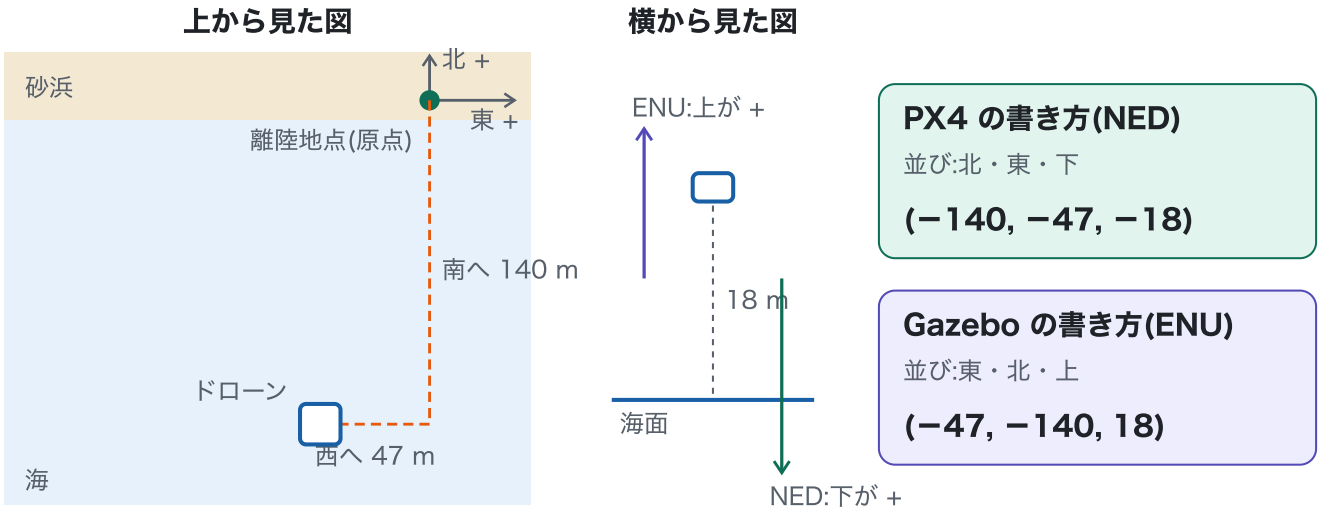


図 8-2 PX4 と Gazebo での同じ位置の書き方

実機での接続

シミュレーションでは、PX4 もエージェントも同じ PC で動くので、UDP でつないでいる。実機では、フライトコントローラと機上コンピュータ(Jetson など)を、シリアル(UART)か Ethernet でつなぐ。エージェントを機上コンピュータで動かせば、mission や detector のコードはそのまま使える。ROS 2 の側から見ると、トピックの名前も型も変わらないからである。

MicroXRCEAgent が止まった場合の動作

MicroXRCEAgent が止まると、mission が送る仮の目標点と「外から操縦するよ」の合図が、PX4 に届かなくなる。PX4 は約 1 秒でこれに気づき、その場で止まる、着陸するなどの安全な動きに切り替える。同時に、ドローンの位置も mission に届かなくなる。

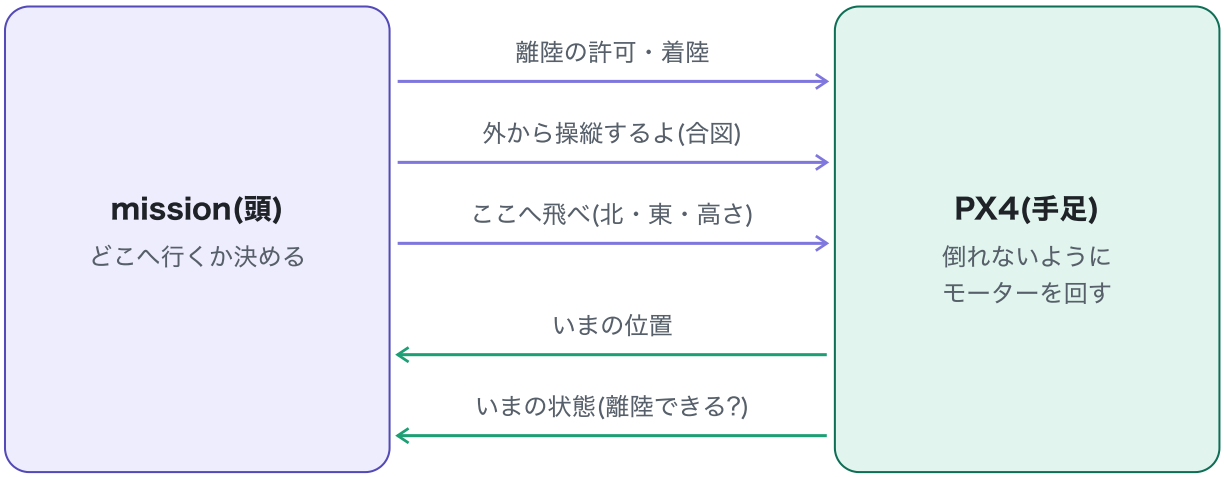
PX4 の版を上げるときの注意

PX4 を新しい版に上げると、mission が位置を受け取れなくなることがある。その場合は、2 つの点を確かめる。1 つは、px4_msgs が新しい PX4 と同じ版でビルドされているかである。もう 1 つは、型の版が変わって、トピック名の後ろの番号(`_v1` など)が変わっていないかである。

8.2 mission と PX4 の役割分担

mission と PX4 がやりとりするものを、図 8-3 に示す。紫の矢印は mission から PX4 へのやりとりで、緑の矢印は PX4 から mission へのやりとりである。

PX4 は、ドローンの中で動く飛行制御のソフトである。風に押されても倒れないように、4 つのモーターの回転を 1 秒に何百回も調整する。mission は「北へ 6 m・東へ 15 m・高さ 18 m の地点へ」と伝えるだけで、モーターをどれだけ回すかは PX4 が決める。



上の 2・3 本目は 0.1 秒ごと(1 秒に 10 回)送り続ける

図 8-3 mission と PX4 のやりとり

8.3 PX4 へのメッセージ

mission から PX4 へ送るメッセージの型は 3 種類ある。それぞれの送り方と、主な項目を表 8-3 に示す。

`VehicleCommand` は関数ではなく、メッセージの型である。「どんな項目を持つデータか」を決めたもので、自分では何も処理しない。PX4 の開発者が配っている `px4_msgs` の中の `VehicleCommand.msg` (テキストファイル)で、項目とお願いの番号表が決まっている。`VehicleCommand()` と書くと、項目が空のメッセージが 1 つできる。それに値を入れて、`publish` で送る。

このプログラムが `VehicleCommand` で出すお願いは 3 つある。その名前と番号、細かい中身、意味を表 8-4 に示す。

メッセージに値を入れて送る仕事は、自分たちで書いた関数 `_cmd` (`mission_node.py` 222 行目)にまとめている。

mission から PX4 へメッセージを送るときに、大事な点が 2 つある。

- **合図は送り続ける。**「外から操縦するよ」がしばらく(初期設定では約 1 秒)届かないと、PX4 は「頭が止まった」と判断し、その場で止まる・着陸するなど、安全な動きに自分で切り替える。そのため 0.1 秒ごとに送っている(166 行目 `create_timer(0.1, …)`)。今回は PX4 の初期設定のままで、頭を止める実験はしていない。
- **高さはマイナスで送る。**PX4 は下をプラスに数えるので、高度 18 m は −18(237 行目の `-alt`)。

表 8-3 mission から PX4 へ送るメッセージの型

メッセージの型	送り方	主な項目と中身
`VehicleCommand`	ときどき 1 回	`command` (お願いの番号)、 `param1` ・ `param2` (細かい中身)
`OffboardControlMode`	0.1 秒ごと	`position` (位置で指示する)、 `velocity` (速さも指示する)
`TrajectorySetpoint`	0.1 秒ごと	`position` (北・東・下)、 `velocity` (北・東・下)、 `yaw` (機首の向き)

表 8-4 このプログラムが出すお願い

名前(番号)	param1, param2	意味
`VEHICLE_CMD_DO_SET_MODE` (176)	1, 6	外から操縦するモード(Offboard)に切り替えて。6 がそのモードの番号
`VEHICLE_CMD_COMPONENT_ARM_DISARM` (400)	1	モーターを回してよい状態にして(アーム)。0 なら止める
`VEHICLE_CMD_NAV_LAND` (21)	なし	その場で着陸して

OffboardControlMode と TrajectorySetpoint のちがい

この 2 つは、どちらも 0.1 秒ごとに送るが、同じことはできない。役割がちがうので、**いつも 2 つセットで**送る。

2 つのメッセージの中身を、図 8-4 に並べる。右の数字は、北東へ秒速 7 m で進んでいるときの例である。

図 8-4 の 2 つのメッセージのちがいを、中身・例・たとえの 3 つの点から、表 8-5 にまとめる。

2 つに分かれている理由

1. **どの欄を使うかを、はっきり伝えるため。** `TrajectorySetpoint` には、位置・速さ・加速度など、たくさんの欄がある。使い方も、「位置だけで指示する」「速さだけで指示する」などいろいろである。そこで `OffboardControlMode` で、どれで指示するかを先に宣言する。PX4 は、「はい」になっている欄だけを使って飛ぶ。
2. **「外から操縦するよ」の合図にもなっているため。**PX4 は、`OffboardControlMode` が届き続けているかを見張っている。上の「大事な点」の「合図は送り続ける」は、このメッセージのことである。

このプログラムでは、どちらも、関数 `_send` (`mission_node.py` 230 行目)で、2 つ続けて送る。

```
mode.position = True                        # 位置で指示する:いつも「はい」
mode.velocity = not math.isnan(vel[0])      # 速さも指示する:速さを渡したときだけ「はい」
...
sp.position = [北, 東, −高さ]               # 位置の数字
sp.velocity = [北, 東, 下]                  # 速さの数字。使わないときは nan(数字なし)
sp.yaw = 機首の向き
```

- **移動しているとき**(にんじん方式、8.4):**位置と速さの両方**で指示する。
- **準備中**のように「その場にいて」と伝えるとき(8.5):**位置だけ**で指示する。速さは nan にして、`velocity` の印も「いいえ」にする。

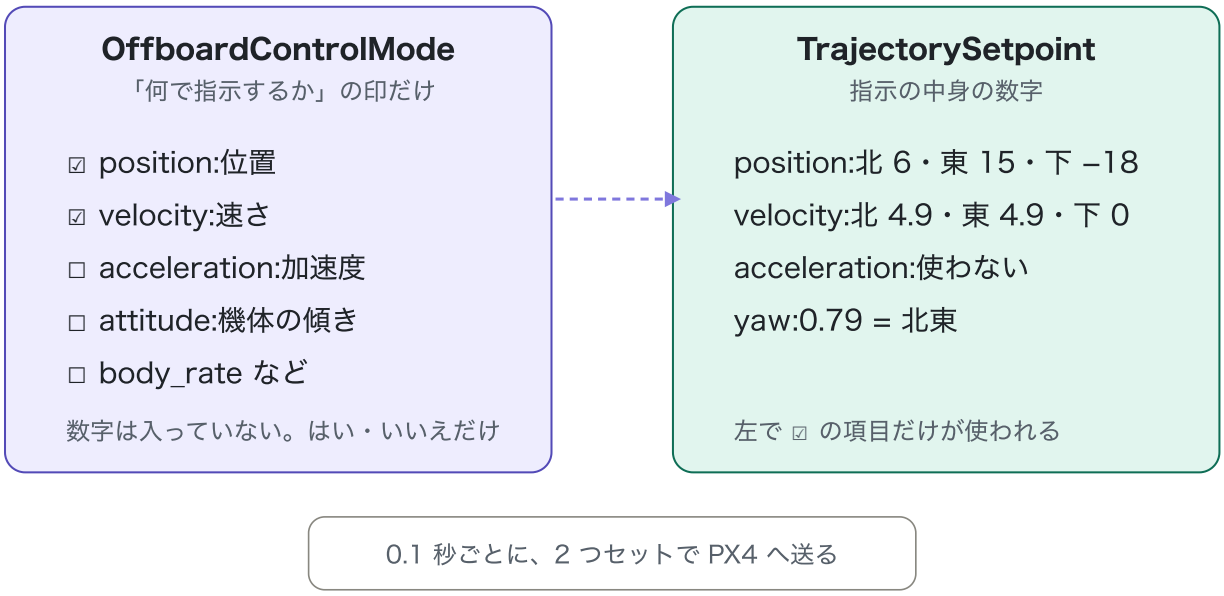


図 8-4 2 つのメッセージのちがい

表 8-5 OffboardControlMode と TrajectorySetpoint

	OffboardControlMode	TrajectorySetpoint
中身	**何で指示するか**の印。はい・いいえだけ	**指示の中身**の数字
例	位置で指示する:はい。速さも指示する:はい	北 6 m・東 15 m・高さ 18 m へ、秒速 7 m で、機首は北東
たとえ	注文票の「どの欄を使うか」に付けるチェック	注文票の欄に書く数字

8.4 仮の目標点による経路追従

馬の前に棒でにんじんをぶら下げると、馬はにんじんを追いかける。にんじんを少しずつ前へ動かせば、馬は同じ速さでついてくる。**馬 = ドローン(PX4)、にんじん = 仮の目標点**である。

にんじん方式でドローンが進む様子を、図 8-5 に示す。図 8-5 は、速さ 10 m/秒で 40 m 先へ進む場合である。オレンジのにんじんは 1 回に 1 m 進み、灰色のドローンは少し遅れて追う。ドローンが半径 1.5 m の緑の円に入ったら、「着いた」とする。ただし、図のドローンの追い方は説明用の簡単な計算であり、PX4 の本当の動きではない。

1. 最初、にんじんはドローンの今の位置に置く。
2. 0.1 秒ごとに、にんじんを目的地の方へ「速さ × 0.1 秒」だけ進め、PX4 に送る。たとえば、移動の 10 m/秒なら 1 回に 1 m、捜索の 7 m/秒なら 0.7 m 進める。
3. にんじんが目的地に着いたら、そこで止める。
4. **ドローン本体**が目的地から 1.5 m 以内に来たら「着いた」とする。にんじんだけが目的地に着いても、ドローンはまだ手前にいるからである。

目的地をいきなり送らないのは、次の理由からである。遠くの目的地をいきなり送ると、PX4 は自分の判断で思いきり加速する。にんじんを少しずつ動かせば、決めた速さ(移動 10 m/秒、捜索 7 m/秒)で、まっすぐ飛べる。人を見落とさないように、決めた速さで飛ぶことが大事である。高さも同じやり方で、1 回に 0.3 m(3 m/秒)ずつ動かす。

仮の目標点の位置と一緒に速さも送るのは、次の理由からである。`_send` は仮の目標点の位置だけでなく、「にんじんが動いている速さ」も送る。位置だけだと、PX4 は追いついては止まり、また追いかける、というぎくしゃくした動きになりやすい。速さも伝えると、PX4 は先回りしてなめらかに飛べる。

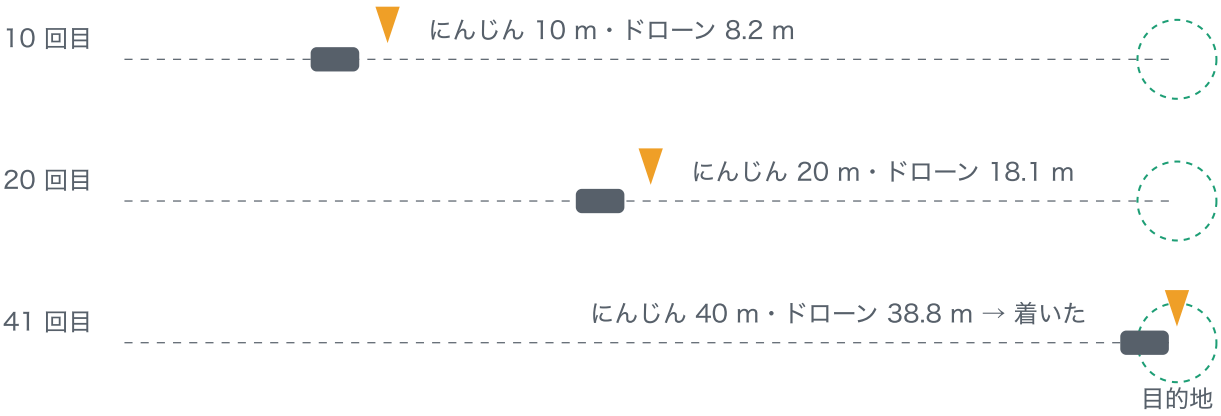


図 8-5 にんじん方式の動き

機首方位の制御

機首の向きは、いつも進む向きと同じとは限らない。場面によって、次の 3 通りがある(`_fly_to` の `yaw`)。

図 8-6 は、ふだんのときと、人を確認・見守りしているときの機首の向きを、上から見て比べたものである。三角の出っぱりが機首である。左の図のように、ふだんは進む向きに機首を向ける。右の図のように、人を確認・見守りしているときは、機首を人に向けたまま、人と一緒に流れる。

3 通りの場面ごとの機首の向きと、その理由を、表 8-6 にまとめる。

そのため、**進む向きと機首の向きがちがう**ことがある。いちばんわかりやすいのは、高度 10 m で人を確認しているときである(図 8-6 の右。見守りの高度 12 m でも同じ)。ドローンは、流されている人と一緒に、海流の向き(南南西)へ秒速 0.25 m でゆっくり動く。でも機首は、人の方向に向けたままである。人が北東にいれば、ドローンは機首を北東に向けたまま、横向きに流れていく。

ここまでの処理が、コードのどこに書いてあるかを、表 8-7 に示す。これらの処理は、`mission_node.py` の 243 行目にある `_fly_to` で行っている。

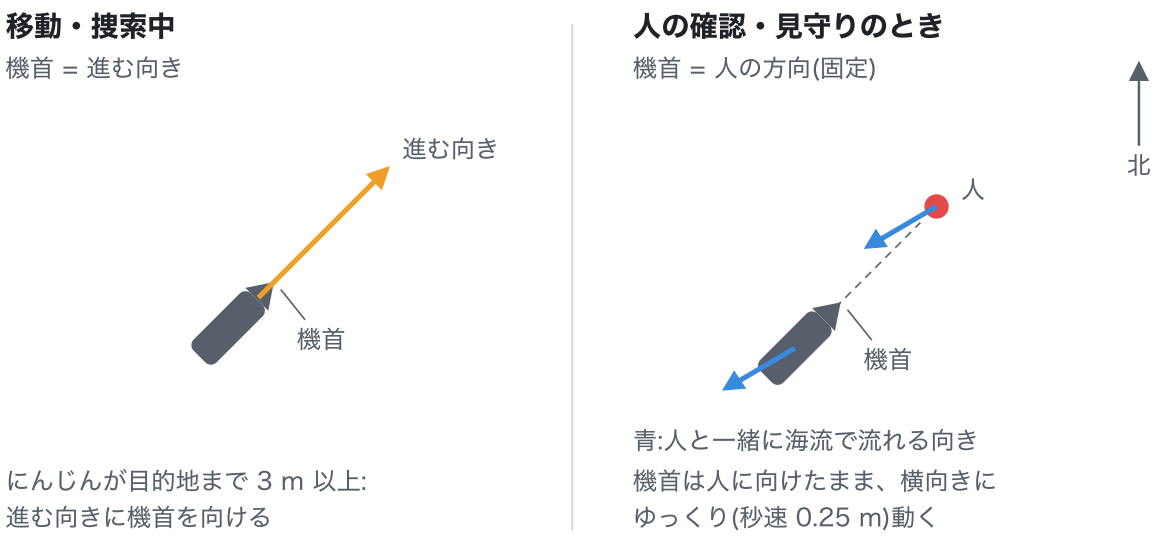


図 8-6 機首の向き

表 8-6 機首の向きの決め方

場面	機首の向き	理由
移動・捜索中で、にんじんが目的地まで 3 m 以上	**進む向き**	カメラは機首の方へ 30° 傾いているので、進む先の海を見られる
にんじんが目的地まで 3 m 未満、または上がる・下がるだけ(離陸、上昇)	**前の向きのまま**変えない	残りが短いと進む向きの計算がぶれて、機体がくるくる回ってしまうため
人の確認・見守りのとき(候補へ移動・降下・確認・オペレータ待ち・見守り)	**確認を始めたときの、人の方向に固定**(`yaw=self.bearing`)	人をカメラの真ん中に写し続けるため

表 8-7 コードとの対応

説明	コード
にんじん	`self.carrot`
1 回に進める長さ	`step = speed * dt` (dt = 0.1 秒)
目的地の方へ進める	`cn += dn / dh * step`
PX4 に送る(合図と、位置・速さ・向きの 2 枚セット)	`self._send(cn, ce, ca, vel)`
にんじんが着いて、本体も 1.5 m 以内なら「着いた」	`(cn, ce, ca) == (n, e, alt) and err < accept_radius`

8.5 離陸シーケンス

mission が起動してから離陸するまでの状態の流れを、図 8-7 に示す。この流れは、`mission_node.py` の 330 行目から書いてある。

図 8-7 の各段階でやることを、表 8-8 にまとめる。



図 8-7 離陸までの状態の流れ

表 8-8 各段階でやること

段階	やること
1. 準備	PX4 から聞いた**今いる位置を、そのまま**仮の目標点として送り続ける(`self._send(self.pos.x, self.pos.y, -self.pos.z)` 。地上ではほぼ北 0・東 0・高さ 0)。約 2 秒(20 回)送ったら、モード切り替えとアームをお願いする
2. 返事待ち	ドローンが今いる位置を送り続けながら、PX4 の「いまの状態」で、Offboard モードになり、しかもアームできたかを確かめる。そろえば今の場所を離陸地点として覚える。3 秒でそろわなければ 1 に戻る
3. 離陸	にんじん方式で真上の高さ 18 m へ(3 m/秒で約 6 秒)。着いたら漂流を予測して捜索の経路を作る

モード切り替えとアーム

準備(INIT)では、モード切り替えとアームを、**別々に 2 つ**お願いする。同じ 0.1 秒の中で、続けて送る(`mission_node.py` 333〜334 行目)。

```
self._cmd(VehicleCommand.VEHICLE_CMD_DO_SET_MODE, 1.0, 6.0)      # お願い 1:モード切り替え
self._cmd(VehicleCommand.VEHICLE_CMD_COMPONENT_ARM_DISARM, 1.0)  # お願い 2:アーム
```

この 2 つのお願いのちがいを、表 8-9 に示す。

どちらか片方だけでは飛べない。モードだけ切り替えても、アームしていなければプロペラは回らない。アームだけしても、Offboard モードでなければ、mission が送る仮の目標点に従わない。

返事も 2 つとも確かめる。返事待ち(ARMING)では、PX4 から届く状態を見て、両方そろったかを確かめる(339〜340 行目)。

```
if (self.nav_state == VehicleStatus.NAVIGATION_STATE_OFFBOARD      # モードが切り替わった
        and self.arming_state == VehicleStatus.ARMING_STATE_ARMED):  # アームできた
```

片方でも断られていれば、3 秒後に準備(INIT)へ戻って、2 つともお願いし直す。

この順番の理由

- **モードを切り替える前に、2 秒間送り続ける。**PX4 は、仮の目標点がもう届いている状態でないと、Offboard モードに切り替えない。切り替えたとたんに指示が来なかったら危ないからである。
- **送るのは、ドローンが今いる位置である。**その理由は、アームした瞬間にドローンが勝手に動き出さないようにするためである。「ここから動かないで」と伝えているのと同じである。
- **お願いを出したあと、PX4 の返事を確かめる。**お願いは断られることがある(準備ができていない、安全チェックに通らないなど)。送っただけで切り替わったと思いこまない。

なお、「20 回」は PX4 からドローンの位置が届く前から数え始めている。ドローンの位置が届くのが遅れると、2 秒送る前にお願いを出すことがある。その場合も、断られたら「3 秒でやり直し」で救われる。実際の飛行でも離陸できていた。

表 8-9 2 つのお願いのちがい

	お願い 1:モード切り替え	お願い 2:アーム
何を決めるか	**だれの指示で飛ぶか**	**モーターを回してよいか**
中身	外から、つまり mission からの指示で飛ぶモード(Offboard)にして	モーターを回してよい状態にして
車でたとえると	運転を自動運転に任せる設定にする	エンジンをかける

8.6 設計上の考慮点

mission が止まった場合の動作

mission が急に止まると、「外から操縦するよ」の合図が PX4 に届かなくなる。PX4 は約 1 秒でこれに気づき、その場で止まる、着陸するなどの安全な動きに自分で切り替える。PX4 自身は動き続けているので、ドローンが落ちることはない。

離陸の準備のやり直し

返事待ち(ARMING)で、3 秒たっても「切り替わった」という返事が来ないときは、mission は準備(INIT)に戻る。そして、もう一度、仮の目標点を送り、モード切り替えとアームをお願いし直す。

9 章 YOLO による人物検出

9.1 検出処理の流れ

YOLO は、写真を 1 回見るだけで「どこに・何が・どれくらいの確かさで」写っているかを答える AI である。使っているのは YOLO11m で、公開の写真集 COCO(80 種類の物)で学習済みのものである。

その前に、写真の「画素」を説明する。写真は、色の付いた小さな四角が、縦と横にすきまなく並んでできている。この小さな四角の 1 つ 1 つを**画素**という。このシステムのカメラの写真は、横に 1280 個、縦に 960 個の画素が並んでいる(図 9-1)。写真の中の位置は、「左から何番目・上から何番目の画素か」で表す。たとえば、10 章の例では、人は左から 800 番目・上から 380 番目の画素に写っている。人の大きさも画素の数で表し、高度 18 m から撮ると、人は 20 画素ほどの大きさにしか写らない。

図 9-1 の左は写真全体で、右は人のまわりを拡大したものである。拡大すると、写真は小さな四角が並んだものだとわかる。

YOLO は、`yolo_detector.py` の中で、次の 4 段階で動く。

YOLO が写真から人を見つけるまでの 4 段階を、図 9-2 に示す。①〜③は `_infer` で行い、④は `detect` の後半で行っている。

図 9-2 の 4 段階で、それぞれ何をするかを、表 9-1 にまとめる。

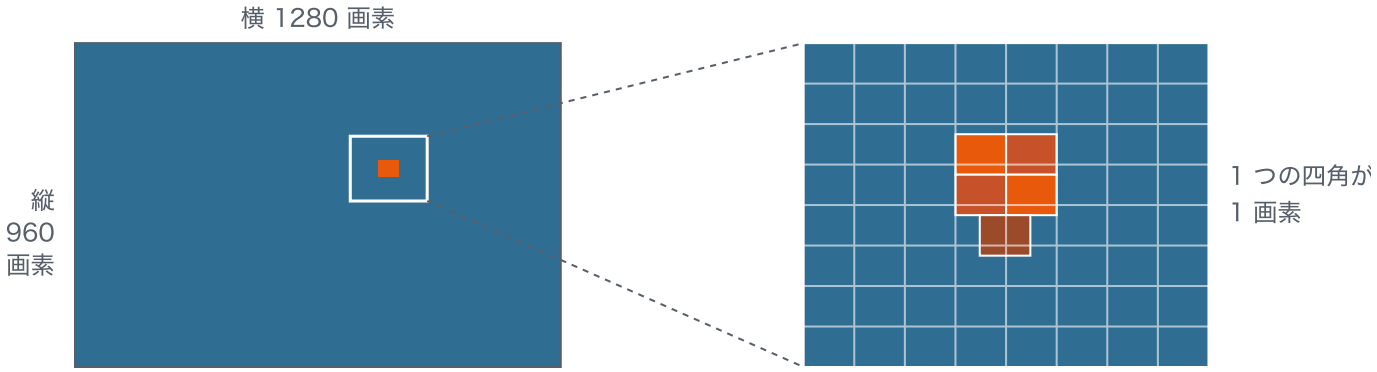


図 9-1 写真と画素

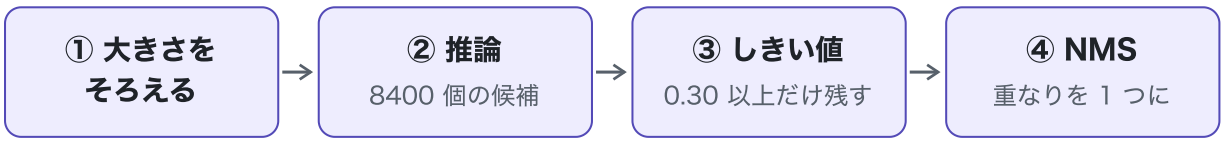


図 9-2 YOLO の 4 段階

表 9-1 4 段階の中身

段階	やること
① そろえる	YOLO は 640 × 640 画素の写真しか受け取れない。縦横の比を変えずに縮小(または拡大)し、余った所を灰色で埋める
② 推論	YOLO が **8400 個の候補**を出す。1 個の候補は 84 個の数字(枠の位置 4 個 + 80 種類それぞれの確信度)
③ しきい値	各候補で一番高い確信度を見て、0.30 未満の候補を捨てる
④ NMS	同じ人に重なって付いた枠を、確信度が一番高い 1 つだけ残して消す

9.2 YOLO の出力

YOLO は、写真を 3 種類の細かさのます目に分けて、ます目ごとに「ここに何かあるか」を答える。

$$80 \times 80 + 40 \times 40 + 20 \times 20 = 6400 + 1600 + 400 = 8400 \text{ 個}$$

細かいます目(80 × 80)は小さい物、粗います目(20 × 20)は大きい物を受け持つ。出力は「84 行 × 8400 列」の表である。コードでは、これを次のように分けている。

```
out = self.sess.run(None, {self.input_name: x})[0][0]     # (84, 8400)
boxes, scores = out[:4].T, out[4:].T     # 上の 4 行 = 枠、残りの 80 行 = 80 種類の確信度
cls = scores.argmax(1)                   # 一番確信度が高い種類の番号(人は 0 番)
conf = scores.max(1)                     # その確信度
```

枠は「中心の x・中心の y・幅・高さ」で出てくるので、あとで「左上・右下」の形に直している。

9.3 NMS

NMS が重なった枠を 1 つにする様子を、図 9-3 に示す。

8400 個の候補のうち、隣り合うます目が同じ人に枠を付けることがよくある。NMS(Non-Maximum Suppression、「一番以外を消す」)で、重なった枠を 1 つにする。重なりの割合は IoU で測る。IoU は、2 つの枠が重なった面積を、2 つの枠を合わせた面積で割った値である。

図 9-3 のように、NMS は、確信度が一番高い枠を残し、その枠と大きく重なる枠を消す。このシステムでは、IoU が 0.45 以上なら、大きく重なっているとみなす。



図 9-3 NMS

9.4 タイル分割推論

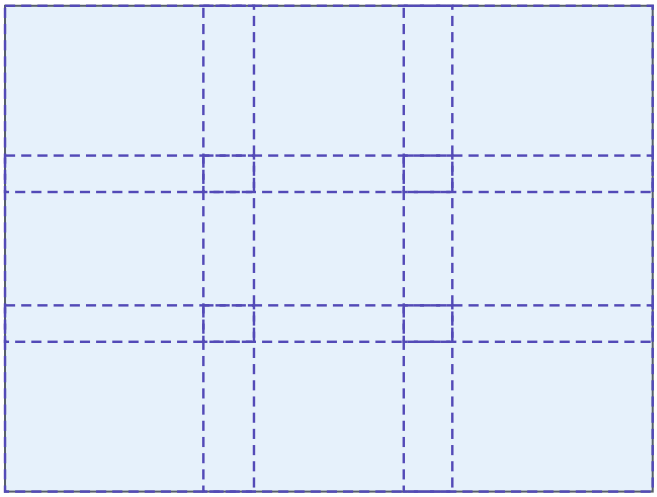
高度 18 m から撮ると、人は写真の中で数十画素にしか写らない。写真全体(1280 × 960)を 640 × 640 に縮めると、人はさらに半分の大きさになり、見落とす。そこで、**写真を重なりのある小さなタイルに分け、タイルごとに推論**している。

3 × 3 のタイルの分け方を、図 9-4 に示す。点線が 1 枚のタイルである。タイルを隣と重ねるのは、タイルの境目にいる人を切らないためである。

分け方によって、人が YOLO にどれくらいの大きさで届くかを、表 9-2 で比べる。表 9-2 は、写真を YOLO に入る大きさの 640 × 640 に合わせたとき、写真の中で 20 画素の人が何画素になるかを計算したものである。

YOLO は、どんな写真も 640 × 640 に直してから見る。大きい写真ほど強く縮められ、人も小さくなる。小さく切ってから見せれば、縮められずに済み、人が大きいまま YOLO に届く。

タイルで見つけた枠は、タイルの左上の位置を足して、元の写真の座標に戻す。重ねた部分では同じ人が 2 枚のタイルで見つかるが、それも次の NMS で 1 つになる。



高度 12 m 以上:3 × 3 = 9 枚

1 枚 493 × 370 画素、隣と 2 割重ねる
640 に合わせて 1.3 倍に(人 20 → 26 画素)

高度 12 m 未満:2 × 2 = 4 枚

1 枚 712 × 534 画素(0.9 倍に縮む)
近いので人が大きく写り、4 枚で足りる

比べると:分けずに全体を 1 回

0.5 倍に縮む(人 20 → 10 画素)

図 9-4 3 × 3 のタイル分割

表 9-2 タイルの分け方と人の写る大きさ

見せ方	640 に合わせるための倍率	人の大きさ
分けずに、写真全体(横 1280)を 1 回	640 ÷ 1280 = 0.5 倍(縮む)	10 画素
2 × 2 に分ける(1 枚 横 712)	640 ÷ 712 = 0.9 倍(少し縮む)	18 画素
3 × 3 に分ける(1 枚 横 493)	640 ÷ 493 = 1.3 倍(大きくなる)	26 画素

9.5 クラス別 NMS の実装

```
105  # クラスごとに NMS(タイルの重なりで生じた重複もここで消える)
106  for k in np.unique(cls):                  # 見つかった種類の番号を 1 つずつ
107      name = COCO_NAMES[int(k)]             # 番号 → 名前(0 → 'person')
108      if name not in TARGET_CLASSES:        # 人と船以外(車・鳥など)は
109          continue                          #   飛ばす
110      m = cls == k                          # この種類の候補だけを選ぶ印
111      b, c = boxes[m], conf[m]              # その枠と確信度
112      xywh = ...                            # 枠を「左上・幅・高さ」の形に直す(NMS の道具の都合)
113      idx = cv2.dnn.NMSBoxes(xywh, c, self.conf_th, self.iou_th)   # NMS。残す枠の番号が返る
114      for i in np.array(idx).flatten():     # 残った枠を 1 つずつ
115          dets.append(Detection(…))         #   結果に加える
117  return dets
```

種類ごとに分けて NMS をするのは、「人の枠」と「船の枠」が重なっていても、どちらも残したいからである(船の上に人がいる場合など)。

なお、ファイルの先頭の説明には「YOLOv8」と書いてあるが、実際に使っているのは YOLO11 である。出力の形(84 × 8400)が同じなので、同じコードで動く。

9.6 確信度のしきい値

確信度のしきい値は、使う場所によって 3 つある。だんだん厳しくしていく。

3 つのしきい値の値と、使う場所を、表 9-3 に示す。

このほかに、NMS の重なりのしきい値 `iou_th = 0.45` がある。これは確信度ではなく、「どれだけ重なったら同じ人とみなすか」の値である。

表 9-3 確信度のしきい値

名前	値	場所	意味
`conf_threshold`	0.30	detector(YOLO)	これより低い枠は捨てる。mission に渡す最低ライン
`candidate_conf`	0.45	mission(捜索中)	これ以上なら「候補」にして、降りて確かめに行く
`confirm_conf`	0.60	mission(確認高度)	高度 10 m で、これ以上が 2 回以上(3 秒以上見て)そろえば「確定」

9.7 設計上の考慮点

写真全体を 1 回で推論した場合の見落とし

高度 18 m で写真全体を 1 回だけ推論すると、1280 × 960 の写真が 640 × 640 に縮められる。そのため、20 画素で写っていた人は 10 画素になり、YOLO は人の形を読み取れなくなる。これが、9.4 節のタイル分割推論を使う理由である。

確信度 0.50 の検出の扱い

しきい値は使う場所によって違うので、同じ確信度でも扱いが変わる。たとえば、確信度 0.50 の検出は、捜索中は 0.45 以上なので候補になり、ドローンは降りて確かめに行く。一方、確認高度では 0.60 未満なので、確定に必要な 1 回には数えられない。

10 章 画像座標から地上座標への変換

YOLO が答えるのは「写真の中のどこに人がいるか」(画素)である。ドローンが人のところへ飛ぶには、「海の上のどこにいるか」(北へ何 m・東へ何 m)が必要である。この章では、その直し方を 1 つの例で追う。

例として、次の場合を考える。ドローンは北 −140 m・東 −47 m、高度 18 m、機首は北東(45°)を向いている。カメラは、真下から機首の方へ 30° 傾けてある。写真の中の人は、横 800・縦 380 画素の位置に写っている。

一言でいうと、**写真の点** → **角度** → **海面までの距離** → **地図の向き**の順に直す。

この章の図の印の意味は、次のとおりである。●(赤く塗った点)は**海の上にいる本当の人**、○(赤い輪)は**写真に写った人**(写真の上の点)である。図によって、見ている向き(横から・上からなど)がちがう。

10.1 画像中心からのずれ

人が写真の真ん中からどれだけずれているかを、図 10-1 に示す。人は、写真の真ん中 (640, 480) から、右へ 800 − 640 = 160 画素、上へ 480 − 380 = 100 画素ずれている。画素の縦の番号は、下へ行くほど大きくなる。従って、上へのずれは、480 から 380 を引いて求める。

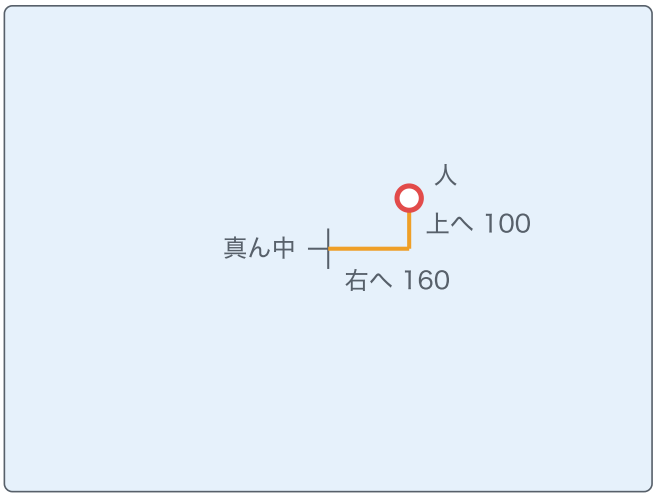


図 10-1 写真の中の人のずれ

10.2 画素から角度への変換

10.1 で、人は写真の真ん中から「右へ 160 画素・上へ 100 画素」にいるとわかった。でも、画素のままでは、海の上の位置は計算できない。そこで、**カメラから見て、真正面から何度ずれた方向に人がいるか**に直す。3 段階で考える。

① カメラの画角

カメラが一度に見える広さ(画角)は、横に 100° である。真正面から左へ 50°、右へ 50° である。これはカメラの性質で、設定ファイルの `hfov: 1.74` (1.74 ラジアン = 100°)がこれである。

② スクリーンのモデル

写真は、カメラの前に立てたスクリーンに、景色が映ったものだと考える。スクリーンの横幅は 1280 画素で、真ん中から右端までは 640 画素である。

写真の右端には、真正面から右へ 50° の方向の景色が写る(①)。そこで、**スクリーンの右端が、カメラからちょうど 50° の方向に見える**ように、スクリーンを置く。そのためには、カメラから 537 画素はなれた所に置けばよい、と計算できる(図 10-2)。

図 10-2 は、カメラとスクリーンを上から見た図である。スクリーンを 537 画素はなれた所に置くと、スクリーンの右端の 640 画素が、ちょうど 50° の方向に見える。

537 は、次のように求める。図 10-2 の右半分は、縦(カメラからスクリーンまで)と横(640 画素)の直角三角形である。角度と縦・横の長さの関係は、**tan(タンジェント)**で表せる。tan 50° ≒ 1.19 は、「角度が 50° のとき、横は縦の 1.19 倍」という意味である。横が 640 なので、縦は 640 ÷ 1.19 ≒ 537 である。

$$\text{スクリーンまでの距離} = 640 \div \tan 50° = 640 \div 1.19 \fallingdotseq 537 \text{ 画素}$$

この 537 画素を**焦点距離**と呼ぶ。カメラと写真の大きさだけで決まるので、どの写真でも同じ値である。

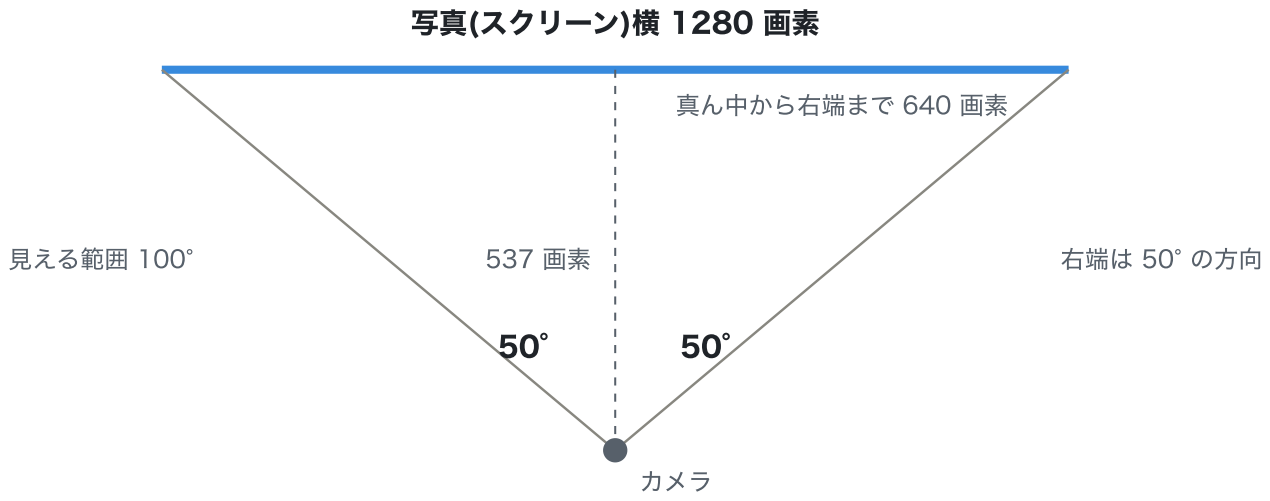


図 10-2 スクリーンまでの距離

③ 人の方向の角度

人は、同じスクリーンの上で、真ん中から右へ 160 画素の所に写っている。カメラから人へ線を引くと、縦 537 画素・横 160 画素の直角三角形ができる(図 10-3)。この三角形の角度が、人のいる方向である。

今度は、縦と横から角度を出す。tan の反対の計算で、**$\tan^{-1}$(アークタンジェント)**と書く。電卓では「横 ÷ 縦」を計算してから $\tan^{-1}$ のボタンを押す。

$$\text{右へ:} \tan^{-1}(160 \div 537) = \tan^{-1}(0.298) \fallingdotseq 16.6°$$

上下も、まったく同じ考え方である。人は真ん中から上へ 100 画素なので、縦 537・横 100 の三角形になる。

$$\text{上へ:} \tan^{-1}(100 \div 537) = \tan^{-1}(0.186) \fallingdotseq 10.5°$$

これで、人は**カメラの真正面から、右へ 16.6° ・上へ 10.5° の方向**にいるとわかった。次の 10.3 では、この方向に線を伸ばして、海面に当たる点を求める。

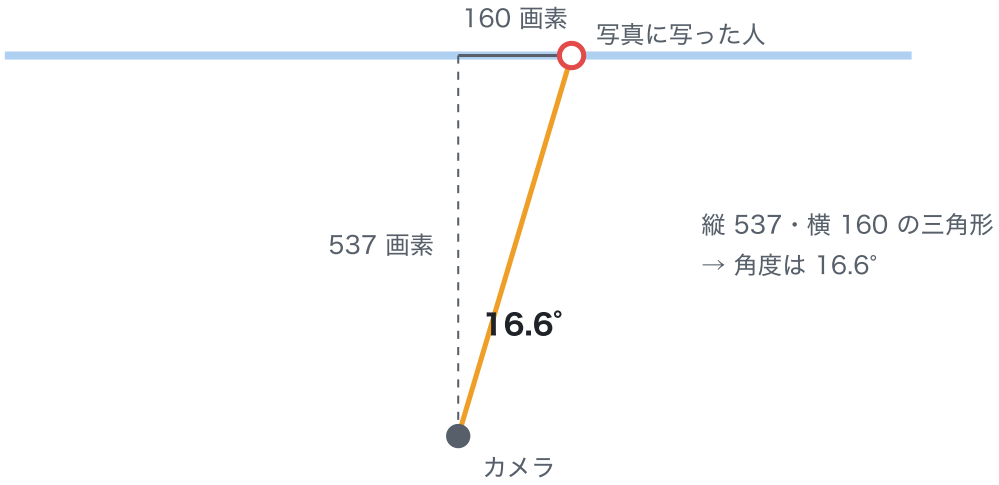


図 10-3 人がいる方向

10.3 視線と海面の交点

カメラから人へ伸ばした線を、横から見た図が図 10-4 である。カメラは真下から 30° 傾いており、人はその真正面から上へ 10.5° の方向にいる。従って、線は真下から 30 + 10.5 = 40.5° の向きに伸びる。真下から 40.5° の線は、前へ 18 × tan 40.5° = 15.4 m 進んだ所で、海面に当たる。

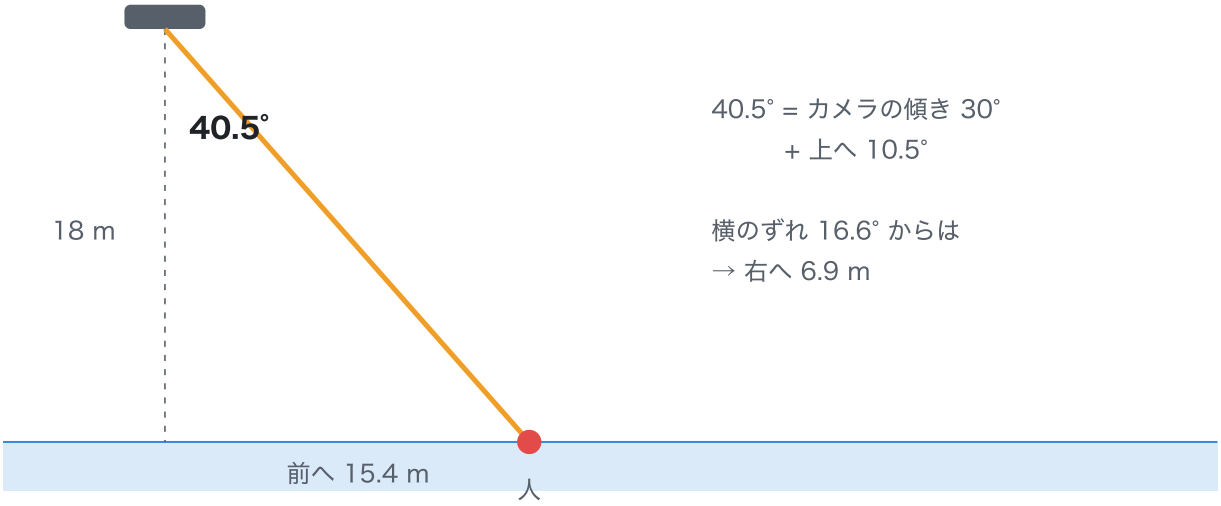


図 10-4 線が海面に当たる点

横方向のずれの計算

ここでやりたいことは、次のことである。写真の上の「右へ 160 画素」を、海の上の「右へ何 m」に直す。

考え方は、次のとおりである。遠くにある物ほど、写真には小さく写る。同じ 160 画素のずれでも、近くなら数十 cm、遠くなら何 m にもなる。だから、**人がカメラからどれだけ遠いか**がわかれば、m に直せる。その「遠さ」が 23.3 m である。

図 10-5 は、10.2 の図 10-3 と同じく、カメラの真正面を上に向けて描いたものである。カメラから人へ伸ばした線(オレンジ)は、どこまで伸ばしても同じ角度(16.6°)である。

図 10-5 は、図 10-3 を、人がいる所まで伸ばしたものである。○ と ● は、同じ線の上にある。青で描いた写真と、緑で描いた人がいる所は、どちらもカメラの真正面の向きに直角である。従って、「先へ行った距離」と「右へのずれ」の割合は、写真の上でも人がいる所でも同じになる。

写真の上では、537 画素先で 160 画素右である。割合は 160 ÷ 537 = 0.30 である。人がいる所でも、割合は同じ 0.30 である。人がいる所が**真正面の向きに 23.3 m 先**なら、右へのずれは次のとおりである。

```
右へ:23.3 × 0.30 ≒ 6.9 m
```

ここで大事なのは、23.3 m を**カメラの真正面の向き(紫の点線)に沿って**測ることである。537 画素も、この向きに沿って測った長さだからである。

図 10-5 の緑の線を立体的に見ると、図 10-6 のようになる。図 10-5 の緑の線は、本当は大きな板である。写真(スクリーン)を、人がいる所まで大きくしたもので、カメラの真正面に直角に立っている(図 10-6)。

図 10-6 の緑の板は、写真を人の所まで大きくした想像上の板で、大きさは横 55.6 m・縦 41.7 m である。板はカメラと一緒に 30° 傾いているので、海面を斜めに横切り、上は空中に、下は海の中に出る。図では、海の中に出た部分を点線で描いた。本当の人 ● は、板と海面が交わる線の上にいて、板の真ん中 D から右へ 6.9 m、上へ 4.3 m の所にいる。この割合は、写真の上の ○ と同じである。

本物の板ではなく、長さを測るために考える板なので、空中にも海の中にも出てかまわない。この板の上では、写真の 1 画素が 23.3 ÷ 537 ≒ 0.043 m(4.3 cm)にあたる。だから、160 画素 × 4.3 cm ≒ 6.9 m である。

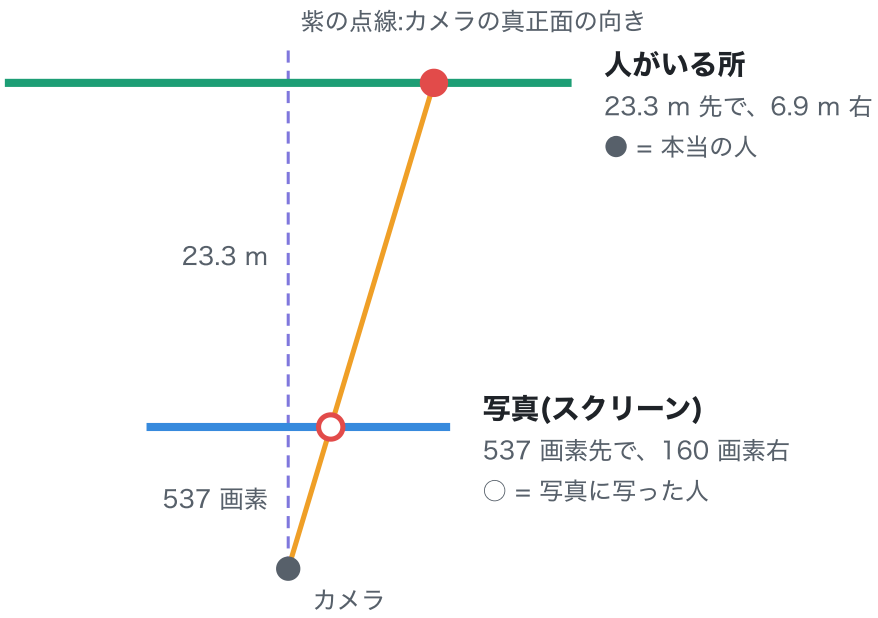


図 10-5 カメラの真正面を上にして見た図

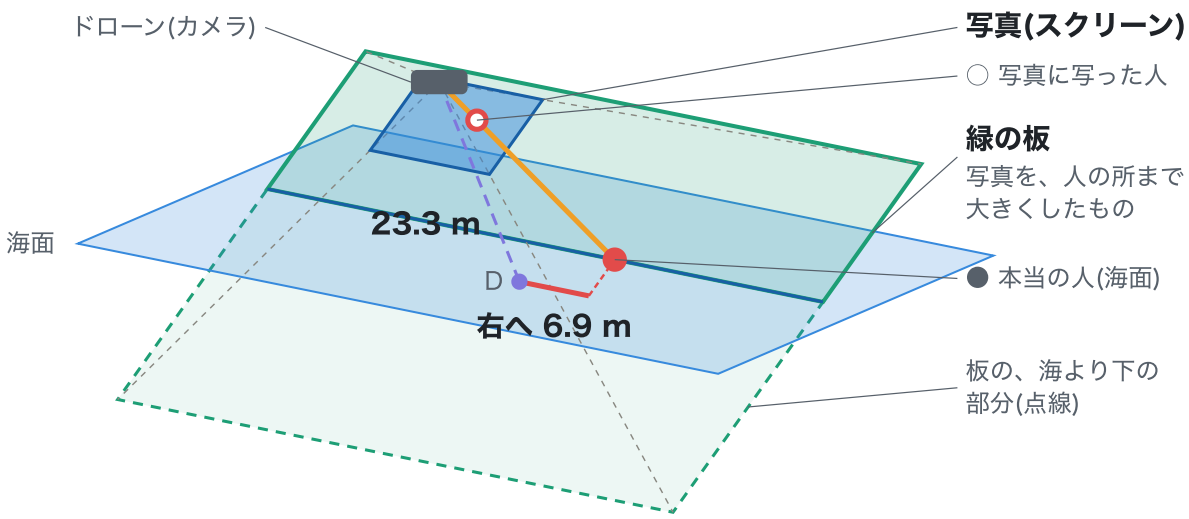
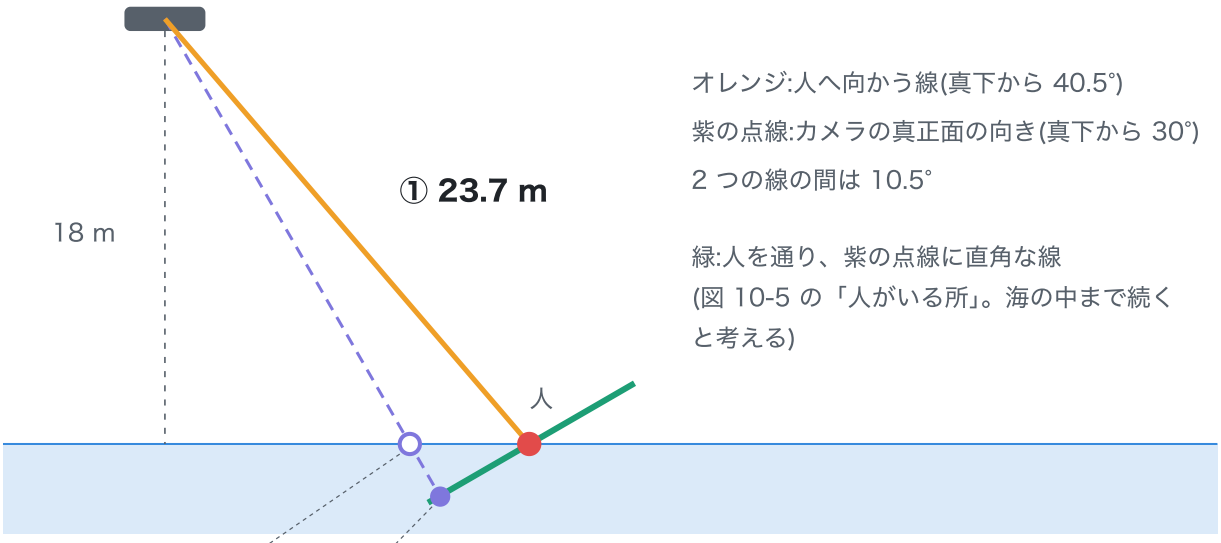


図 10-6 立体的に見た図

奥行き 23.3 m の計算

人がいる所(緑の線)が、カメラの真正面の向き(紫の点線)に何 m 先かを、横から見た図で計算する(図 10-7)。

図 10-7 では、紫の点線を、海面に当たった C で止めずに、緑の線に当たる D まで伸ばしている。そのため、D は海面より下になる。



C:紫の点線が海面に当たる所。カメラから 20.8 m(= 18 ÷ cos 30°)

D:紫の点線が緑の線に当たる所。カメラから 23.3 m ← 求めたいのはこれ

図 10-7 23.3 m の求め方

18 ÷ cos 30° を使わない理由

18 ÷ cos 30° = 18 ÷ 0.866 ≒ 20.8 m は、紫の点線が**海面に当たる所(C)**までの長さである。C は、写真の**ちょうど真ん中**に写っている海の点である。人は写真の真ん中より上に写っている(上へ 100 画素)ので、C より**奥**にい

る。だから、人を通る緑の線は、C より先で紫の点線と交わる(D)。必要なのは D までの長さである。

D までの長さが必要な理由は、次のとおりである。10.2 の「537 画素先で 160 画素右」という割合は、写真と同じ向きの面(紫の点線に直角な面)の上でだけ使える。人がいるのは緑の線の上である。C を通る面は人を通らないので、C までの長さ(20.8 m)では、人の右へのずれは出せない。

D までの長さは、そのままでは測れない。わかっているのは、オレンジの線(カメラから人へ向かう線)の角度である。そこで、次の 2 段階で求める。

1. 1 段階目では、カメラから人まで、オレンジの線に沿った長さを求める。高さ 18 m を、真下から 40.5° の向きに進むと、18 ÷ cos 40.5° = 18 ÷ 0.760 ≒ **23.7 m** で人(海面)に着く。
2. 2 段階目では、そのうち紫の点線の向きに進んだ分を求める。緑の線は紫の点線に直角なので、人から紫の点線へ直角に下ろした所が D である。オレンジの線は紫の点線から 10.5° ずれているので、紫の向きに進んだ分は 23.7 × cos 10.5° = 23.7 × 0.983 ≒ **23.3 m** である。

2 の考え方は、「北から 10.5° 東へずれた向きに 23.7 m 歩くと、北へは 23.3 m 進んだことになる」のと同じである。cos 10.5° = 0.983 は、「10.5° ずれた向きに 1 m 進むと、もとの向きには 0.983 m 進む」という意味である。

コード(`pixel_to_ground`)では、この 2 段階をまとめて計算していて、23.3 m は変数 `t` の値(23.29)にあたる。

10.4 機体座標から北・東への変換

ドローンから見た「前・右」を、地図の「北・東」に直す様子を、上から見た図で図 10-8 に示す。北東を向いたドローンから見て「前 15.4 m・右 6.9 m」の所は、地図の上では「北へ 6.0 m・東へ 15.8 m」の所になる。

```
北 = 前 × cos 45° − 右 × sin 45° = 15.4 × 0.707 − 6.9 × 0.707 = 6.0 m
東 = 前 × sin 45° + 右 × cos 45° = 15.4 × 0.707 + 6.9 × 0.707 = 15.8 m
遭難者の位置 = (−140 + 6.0, −47 + 15.8) = 北 −134.0 m・東 −31.2 m
```

この結果は、コード(`geo.py` の `pixel_to_ground`)で同じ条件を計算した値と一致する。

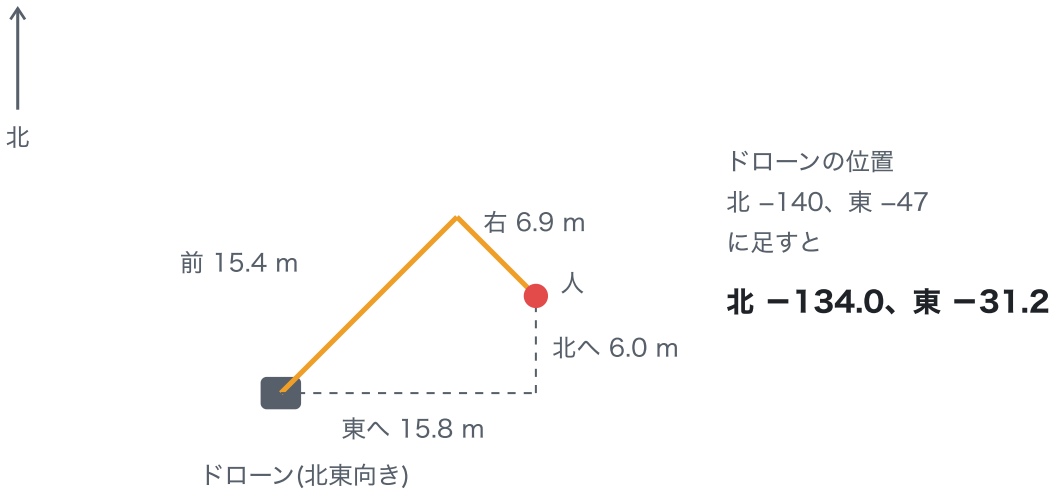


図 10-8 「前・右」を「北・東」に直す

10.5 機体姿勢の補正

飛んでいるドローンは、前に進むときに機首を少し下げる。カメラは機体に固定されているので、カメラも一緒に下を向く。この傾きを直さないと、計算した遭難者の位置がずれる。

機体が傾くと、線が海面に当たる点がどれだけずれるかを、図 10-9 に示す。機体が前後に 5° 傾くだけで、当たる点は 3 m 近くずれる。このずれは、ボードの停止距離 2 m より大きいので、無視できない。

コードでは、10.4 の「向きで回す」ときに、PX4 から届く**姿勢(四元数 `q`)**を使っている。`q` には、機首の向きだけでなく、前後・左右の傾きも入っている。そのため、向きと一緒に傾きも直る。

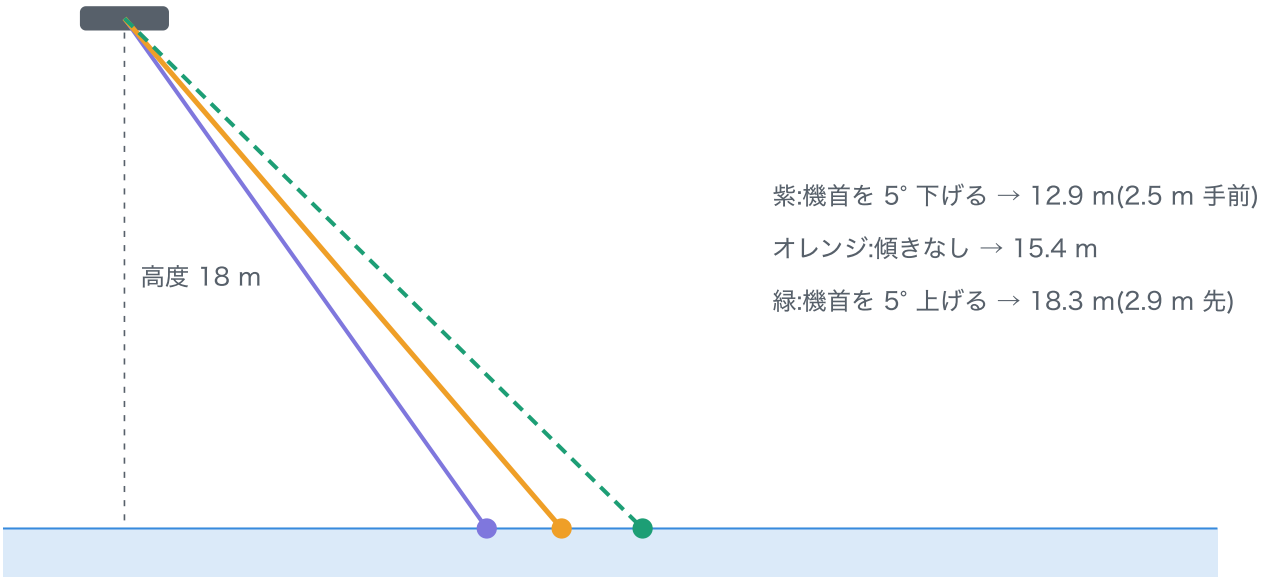


図 10-9 機体の傾きによるずれ

機首を 5° 下げた場合の計算例

10 章の例と**同じ写真**(人は横 800・縦 380 画素の位置)で、撮ったときだけ機首が 5° 下がっていたとする。カメラは機体に固定されているので、カメラも 5° 下を向く。カメラの傾きが、真下から 30° ではなく **25°** になったのと同じである。

傾きなしの場合と、機首を 5° 下げた場合を、横から見た図で図 10-10 に比べる。同じ写真の同じ位置に人が写っていても、機首が 5° 下がっていると、人は 2.5 m 手前にいることになる。

あとは、10.2〜10.4 と同じ手順で、30° を 25° に変えて計算するだけである。

2 つの場合の計算を、手順ごとに表 10-1 に並べる。

機首が下がると、人は 2.5 m 手前にいることになり、北・東の値もそれだけずれる。もし傾きを無視して「傾きなし」の列で計算すると、遭難者の位置を 2.5 m まちがえる。コードは PX4 から届く姿勢 `q` を使うので、自動的に右の列の計算になる。

表の値は、途中の値を四捨五入して書いているので、最後の桁が少しずれることがある。最後の行は、コード `geo.py` で計算した値である。

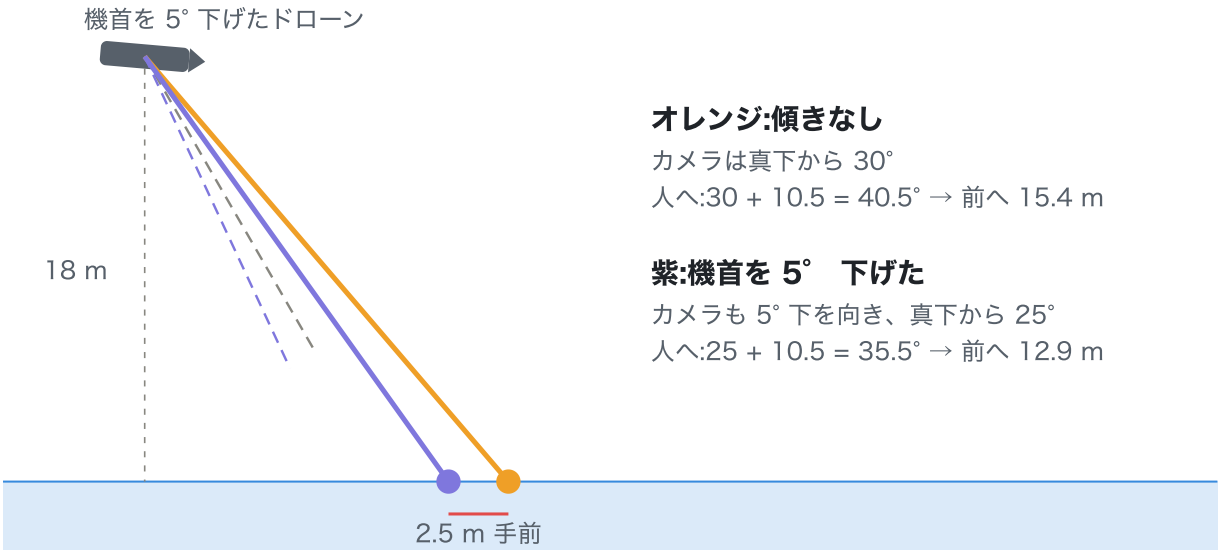


図 10-10 機首を 5° 下げた場合

表 10-1 傾きなしと、機首を 5° 下げた場合の比べ

手順	傾きなし(カメラ 30°)	機首 5° 下げ(カメラ 25°)
9.2 写真から角度	右へ 16.6°・上へ 10.5°(同じ写真なので同じ)	
9.3 真下からの角度	30 + 10.5 = 40.5°	25 + 10.5 = 35.5°
9.3 前へ	18 × tan 40.5° = 15.4 m	18 × tan 35.5° = 12.9 m
9.3 真正面に沿った距離(緑の板まで)	18 ÷ cos 40.5° × cos 10.5° = 23.3 m	18 ÷ cos 35.5° × cos 10.5° = 21.7 m
9.3 右へ	23.3 × 0.298 = 6.9 m	21.7 × 0.298 = 6.5 m
9.4 北へ(前 × 0.707 − 右 × 0.707)	(15.4 − 6.9) × 0.707 = 6.0 m	(12.9 − 6.5) × 0.707 = 4.5 m
9.4 東へ(前 × 0.707 + 右 × 0.707)	(15.4 + 6.9) × 0.707 = 15.8 m	(12.9 + 6.5) × 0.707 = 13.7 m
9.4 ドローンの位置(北 −140・東 −47)を足す	**北 −134.0・東 −31.2**	**北 −135.5・東 −33.3**

10.6 緯度・経度への変換

「北へ何 m・東へ何 m」は、離陸地点から見た距離である。救助隊に知らせるときは、緯度・経度に直す(`ned_to_latlon`)。離陸地点は北緯 26.5472°・東経 128.0540° としている。

例で求めた遭難者の位置、北 −134.0 m・東 −31.2 m を、緯度・経度に直す計算を表 10-2 に示す。

表 10-2 北 −134.0 m・東 −31.2 m を緯度・経度に

	1° の長さ	計算	結果
緯度	約 111.3 km(どこでも同じ)	134.0 ÷ 111,319 = 0.00120° 減る	26.545996°
経度	約 99.6 km(この緯度では)	31.2 ÷ 99,583 = 0.00031° 減る	128.053687°

経度 1° の長さ

北緯 26.5° は、今回のドローンが飛んでいる場所の緯度である。離陸地点を北緯 26.5472° と決めているので、その緯度での経度 1° の長さを使う。コードの `ned_to_latlon` も、離陸地点の緯度 `lat0` を使って `R_EARTH * cos(radians(lat0))` と計算している。飛ぶ場所が変われば、この長さも変わる。たとえば東京(北緯 35.7°)では経度 1° は約 90.4 km、札幌(北緯 43.1°)では約 81.3 km である。

経度 1° の長さが緯度によって変わる理由を、図 10-11 に示す。左は、地球を横から見た図である。経線は北極に向かって集まるので、経度の差が同じでも、北へ行くほど、赤で示した東西の幅が短くなる。右は地球の断面で、北緯 26.5° の東西の輪は、赤道の輪より小さい。

- **経度**は、東西にぐるっと一周する輪を、360 に分けた目もりである。だから、経度 1° の長さ = その輪の一周 ÷ 360 である。
- **赤道の輪**は一番大きく、一周 40,075 km である。経度 1° = 40,075 ÷ 360 ≒ **111.3 km**。
- **北緯 26.5° の輪**は、それより小さくなる。図の右のように、輪の半径は 6,378 km × cos 26.5° = 5,706 km で、赤道の 0.895 倍である。一周も 0.895 倍の 35,850 km なので、経度 1° = 35,850 ÷ 360 ≒ **99.6 km**(= 111.3 × 0.895)。
- **緯度**の 1° は、北極と南極を通る大きな輪(どれも赤道と同じ大きさ)を 360 に分けた目もりなので、どこでも **111.3 km** である。

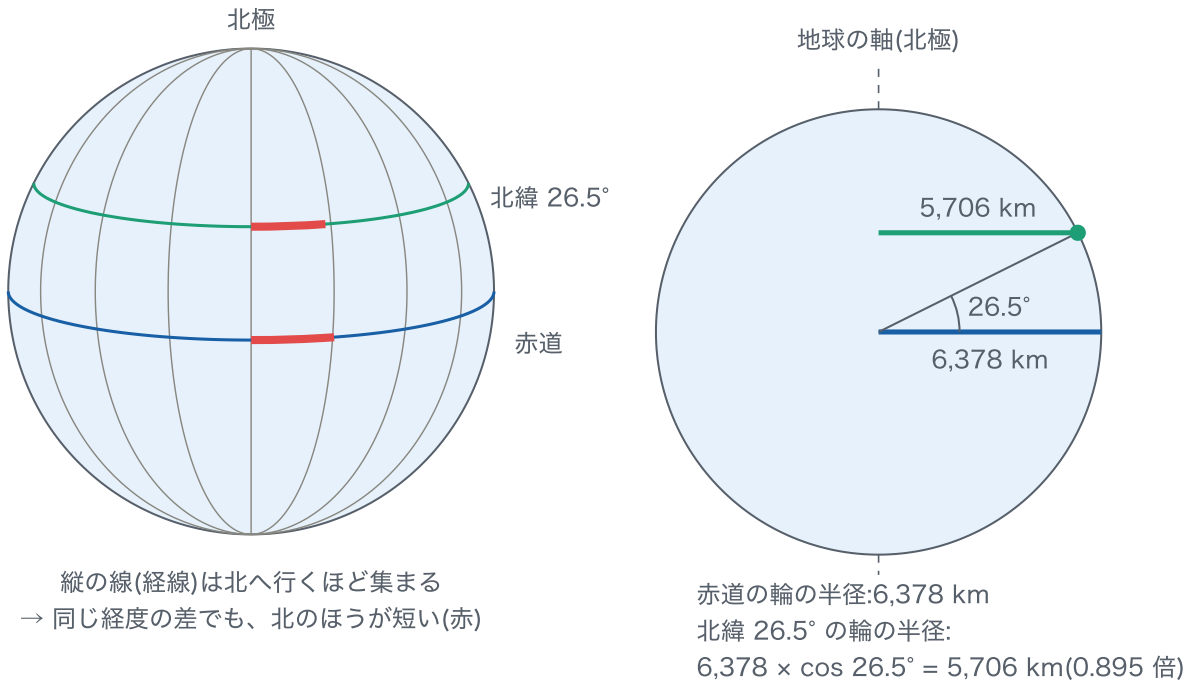


図 10-11 経度 1° の長さが緯度で変わる理由

10.7 コードとの対応

10.1〜10.6 の各段階が、`geo.py` のどの関数にあたるかを、表 10-3 に示す。

コードは、「前・右・下」の 3 つの数の組(ベクトル)で、10.2〜10.4 をまとめて計算している。この資料では、わかりやすくするために、横から見た図と上から見た図に分けて説明した。

表 10-3 計算の段階とコードの対応

段階	コード(geo.py)
9.1・9.2 画素 → 角度(とカメラの傾き)	`pixel_ray_body` 。焦点距離は `f = (width / 2) / tan(hfov / 2)`
9.4・9.5 向きと傾きで回す	`quat_to_rot(q) @ …` (`@` は「回転の表を掛ける」計算)
9.3・9.4 海面に当てる、位置を足す	`pixel_to_ground` の後半(`t = (ground_z − pos_ned[2]) / ray[2]`)
9.6 緯度・経度	`ned_to_latlon`

10.8 設計上の考慮点

傾きを補正しない場合のずれの向き

ドローンが機首を 10° 下げているのに、傾きを直さずに計算すると、遭難者の位置は実際より手前にずれる。機首が下がるとカメラも下を向くので、線が近くの海面に当たるからである。線の角度は真下から 30.5° になるので、当たる点は 18 × tan 30.5° = 10.6 m 先となり、本当の 15.4 m より約 5 m 手前になる。

経度の数字が大きく変わる理由

同じ 100 m 進んでも、経度の数字は緯度の数字より大きく変わる。この緯度では、経度の 1° が 99.6 km であり、緯度の 1° の 111.3 km より短いからである。

11 章 mission の状態遷移

mission(`mission_node.py`)は、0.1 秒ごとに `_on_timer` (316 行目)を呼ばれる。そのたびに、今の状態(`self.state`)に応じて 1 段階だけ仕事をし、条件がそろったら次の状態へ切り替える(`_goto`)。この章では、捜索から見守りまでの流れをコードで追う。離陸までは 8.5 節で説明した。

11.1 状態遷移図

mission の状態の流れを、図 11-1 に示す。確認の状態を紫で、人の判断をオレンジで、見守りの状態を緑で表した。候補が見つかると、mission は、どの捜索の段階からでも確認に入る。

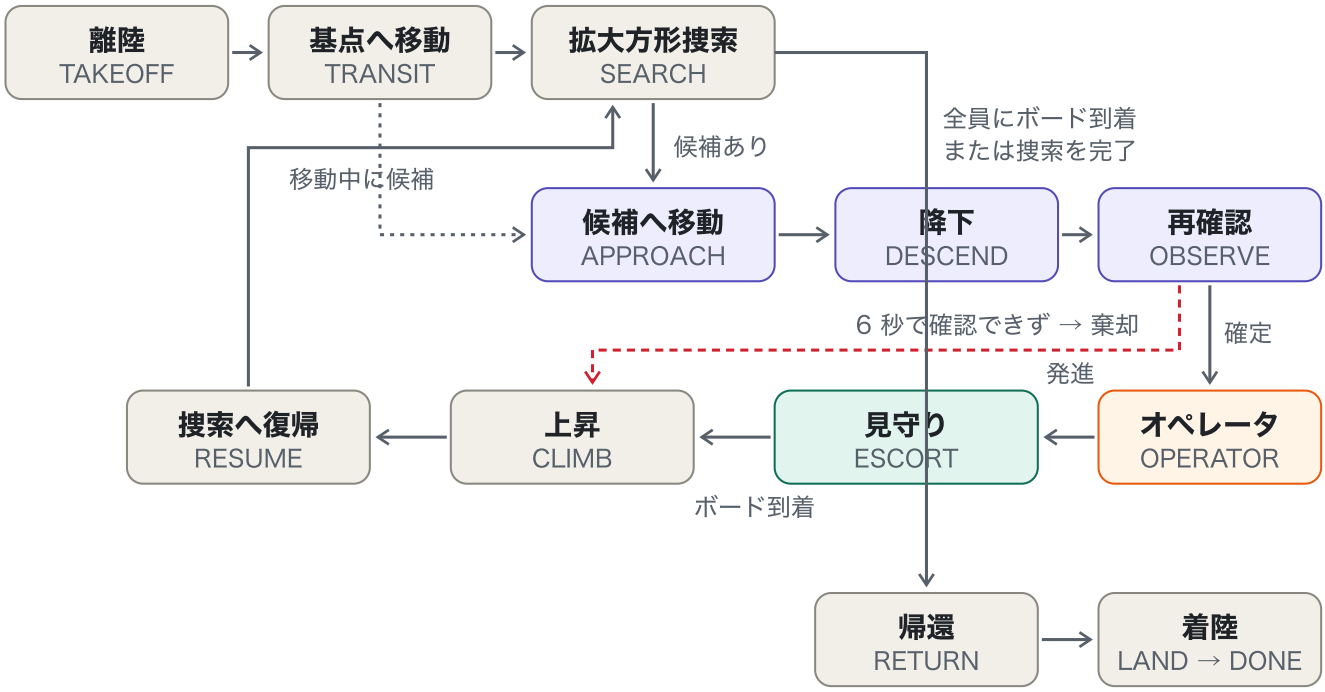


図 11-1 mission の状態の流れ

11.2 検出結果の振り分け

この処理は、 `mission_node.py` の 192 行目にある `_on_detections` で行う。

detector から検出が届くと、今の状態によって 2 通りに使い分ける。

今の状態ごとの検出の使い方を、表 11-1 にまとめる。

`_add_candidate` (207 行目)は、次の 2 つを捨ててから、新しい候補 T を作る。

1. 1 つ目は、北 −8 m より陸側の検出である。これは砂浜と、水際で待っているボードの周りである。浜にいる人やボードの上の物を、遭難者と間違えないためである。
2. 2 つ目は、前に見つけた候補の予測位置から 8 m 以内の検出である。この検出は、前に見つけた候補と同じ人とみなして、新しい候補にはしない(`merge_radius`)。棄却した候補もここで比べるので、同じ木材に何度も降りることはない。

候補は `pending` (未確認)の状態で並ぶ。捜索中は、0.1 秒ごとに `_next_pending` (291 行目)が、未確認の候補のうち**ドローンに一番近いもの**を選ぶ。あれば、すぐ確認に入る(`_start_inspection` 、476 行目)。

```
def _start_inspection(self, tg):
    if self.resume_point is None or self.state in ('SEARCH', 'TRANSIT'):
        self.resume_point = (self.pos.x, self.pos.y)    # 捜索を中断した地点を覚える
    self.current = tg
    pn, pe = tg.predict(time.time(), self.vn, self.ve)
    self.bearing = math.atan2(pe - self.pos.y, pn - self.pos.x)   # 人の方向。確認と見守りの間、機首をこの向きに固定する
```

表 11-1 検出の使い方

今の状態	使い方
再確認・見守り(OBSERVE・ESCORT)	今見ている人の**予測位置から 5 m 以内**の検出だけを、その人の観測(`tg.obs`)に加える。ほかの検出は使わない
捜索・移動・上昇など	確信度 0.45 以上の検出を**候補**にする(`_add_candidate`)

11.3 確認の状態

確認では、状態が APPROACH → DESCEND → OBSERVE の順に進む。この処理は、 `mission_node.py` の 382〜416 行目に書いてある。

確認は 3 つの状態に分かれる。それぞれの状態の目的地と、次の状態へ進む条件を、表 11-2 に示す。

`_standoff` (274 行目)は、遭難者の予測位置から、機首の向き(`self.bearing`)の逆へ「高度 × tan 30°」だけ戻った点を返す。これで、カメラの中心が人に向く。

確定位置は、次のように求める。OBSERVE の間、AI は 1 秒に 1 回、遭難者の位置を測る。確定するときは、確信度 0.60 以上の観測を**今の時刻にそろえてから**平均する。

```
ns = [o[0] + self.vn * (now - o[3]) for o in strong]   # 測った北 + 海流 × 測ってからの秒数
es = [o[1] + self.ve * (now - o[3]) for o in strong]
tg.north, tg.east, tg.t_est = sum(ns) / len(ns), sum(es) / len(es), now
```

2 秒前に測った値は「2 秒前にいた場所」なので、そのまま平均すると、人が流れてくる前の場所が混ざる。そこで、どの値にも「海流 × 測ってからの秒数」を足して、今いるはずの場所に直してから平均する。海流 0.25 m/秒なら、2 秒前の値には 0.5 m を足すことになる。

表 11-2 確認の 3 つの状態

状態	目的地	次へ進む条件
APPROACH	遭難者の予測位置の手前 10.4 m、高度 18 m(`_standoff`)	2.5 m 以内に着いた
DESCEND	遭難者の予測位置の手前 5.8 m、高度 10 m	2.5 m 以内に着いた。観測をいったん空にする
OBSERVE	同じ(人と一緒に流れる)	3 秒以上たって、確信度 0.60 以上の観測が 2 回以上 → 確定。6 秒たっても足りない → 棄却

11.4 オペレータ待ちの状態

オペレータの判断を待つ状態を、OPERATOR と呼ぶ。この処理は、 `mission_node.py` の 418〜435 行目に書いてある。

```
if elapsed_state >= c['operator_delay_s']:                          # 確定から 3 秒たった
    idle = [b for b in self.boards.values() if b['status'] == 'IDLE']   # 砂浜で待っているボード
    if not idle:
        ...「待機中のボードがありません」と記録して待つ
    pn, pe = tg.predict(now, self.vn, self.ve)
    b = min(idle, key=lambda b: math.hypot(b['north'] - pn, b['east'] - pe))   # 人に一番近いボード
    tg.board, tg.dispatch_t = b['id'], now
    self._send_lora('dispatch', tg, now)                            # ボードの発進を LoRa で送る
    self._goto('ESCORT')
```

ボードの状態は、board ノードが 0.2 秒ごとに `/rescue/boards` へ流している。mission はそれを `self.boards` に覚えておき、待っているボード(`IDLE`)を探す。3 台とも発進していたら、どれかが空くまで待つ。今回の想定は 3 名・3 台なので、待つことはない。

この間も、mission は高度 10 m で人を画面の中央に捉えている。映像を地上へ送っている想定の時間は、`self.video_s` に 0.1 秒ずつ足して数えている(327〜328 行目)。足すのは、APPROACH・DESCEND・OBSERVE・OPERATOR の 4 つの状態の間だけである。

11.5 見守りの状態

見守りの状態を、ESCORT と呼ぶ。この処理は、 `mission_node.py` の 437〜455 行目に書いてある。

```
tg = self.current
if tg.obs:
    last = tg.obs[-1]
    tg.north, tg.east, tg.t_est = last[0], last[1], last[3]      # 最新の観測で推定位置と時刻を置き換える
n, e = self._standoff(tg, c['escort_alt'], now)                  # 手前 6.9 m、高度 12 m
self._fly_to(n, e, c['escort_alt'], 4.0, yaw=self.bearing, tol=3.0)
if now - self._last_lora >= c['lora_update_s']:                   # 前に送ってから 1 秒たった
    self._send_lora('update', tg, now)
b = self.boards.get(tg.board)
if b and b['status'] == 'HOLD' and b['target'] == tg.id:          # ボードが止まった
    tg.arrive_t = b['t_arrive']
    tg.status = 'confirmed'
    self._goto('CLIMB')
```

- 見守りの高度は 12 m にした。確認の 10 m より少し高くし、近づいてくるボードも画面に入りやすくした。高度 12 m 以上なので、AI は 3×3 分割で推論する。
- 見守り中は、推定位置を平均せず、最新の 1 回の観測で置き換える。ボードは毎秒新しい位置を受け取るので、1 回ごとの小さなずれは次の更新で直る。平均すると、古い値が混ざって遅れる。
- **送るのは推定位置・推定時刻・海流**(`_send_lora` 、185 行目)。今の予測位置ではなく、推定した時点の値と時刻を送る。ボードが自分で「今」の位置を予測するので、LoRa の遅れ(約 86 ms)の分も自動で補われる。

ボードが止まると、CLIMB で高度 18 m に戻り、RESUME で捜索を中断した地点へ戻ってから、拡大方形捜索の続きを飛ぶ。SEARCH と RESUME の間、0.1 秒ごとに「ボードが着いた人の数が、通報された 3 名に達したか」を確かめ、達したら帰還する(367〜369 行目)。

11.6 設計上の考慮点

見守り中の見間違いの影響

見守り中に、AI が隣の木材を人と見間違えることがある。木材が予測位置から 5 m より離れていれば、観測に加えないので、推定位置は変わらない。ただし、5 m 以内だと推定位置がずれる。今回の想定では、見守り中にそうなったことはない。

確定からボードの発進までの時間

確定からボードの発進までは、最短で 3 秒かかる。この時間は `operator_delay_s` で決めている。報告書では、3 回とも 3.1 秒だった。mission は 0.1 秒ごとにしか判定しないので、少し長くなる。

ボードが足りない場合

3 人目を見つける前に、ボードが 3 台とも発進していた場合は、mission は OPERATOR の状態のまま上空で待つ。そして、どれかのボードが空くまで、次のボードを発進させない。ただし、到着したボードは浮き具として遭難者のそばに残る設計である。従って、実際には、4 人目以降のためのボードを別に用意する必要がある。

12 章 LoRa 通信の模擬

```
ドローン(mission) → /rescue/lora/tx → [lora_link] → /rescue/lora/rx → ボード(board)
```

本物の LoRa 無線の「遅れる」「ときどき届かない」というくせをまねている。lora_link は、届いたメッセージをそのまま渡さずに、遅らせたり、捨てたりしてから渡す。

12.1 送信時間の計算

LoRa は、遠くまで届くかわりに、とてもゆっくり送る無線である。送る時間は、Semtech 社の資料(AN1200.13)の式で計算できる(`lora_airtime_s` 、20 行目)。

SF7・帯域幅 125 kHz で 24 バイトを送るときの計算を、表 12-1 に示す。

③ の 16 は誤り検出(CRC)あり、(1 + 4) は符号化率 4/5 を表す。SF(拡散率)を大きくすると、1 つの記号が 2 倍ずつ長くなり、遠くまで届くが時間がかかる。

SF を変えると、送信時間がどう変わるかを、表 12-2 に示す。値は、24 バイトを帯域幅 125 kHz で送る場合である。

SF12 では 1 回に 1.48 秒かかるので、1 秒ごとに送ることはできない。海水浴場の範囲(数百 m)なら SF7 で届くと考え、SF7 にした。

表 12-1 24 バイトを送る時間の計算

段階	計算
① 1 つの記号(シンボル)を送る時間	2^{SF} ÷ 帯域幅 = 128 ÷ 125,000 = 1.024 ms
② 前置き(プリアンブル)	(8 + 4.25) 記号 = 12.25 記号
③ データの記号の数	8 + ⌈(8 × 24 − 4 × 7 + 28 + 16) ÷ (4 × 7)⌉ × (1 + 4) = 8 + ⌈208 ÷ 28⌉ × 5 = 8 + 8 × 5 = 48 記号
④ 合計	(12.25 + 48) × 1.024 ms = **61.7 ms**

表 12-2 SF による送信時間の違い

SF	7	8	9	10	11	12
送信時間	61.7 ms	113 ms	206 ms	371 ms	823 ms	1483 ms

12.2 パケット長の計算

この処理は、 `lora_link_node.py` の 57 行目にある `_on_tx` で行う。

```
payload = 4 + 4 * sum(1 for v in d.values() if isinstance(v, (int, float)))
```

シミュレーションの中では、メッセージは JSON の文字列で、数十〜百バイトある。実機では、必要な数値だけをバイナリに詰めて送る。そこで、JSON の中の**数値の項目**(北・東・推定時刻・海流の北・東の 5 つ)を 4 バイトずつと数え、種類・宛先・対象をまとめた 4 バイトを足して、24 バイトとしている。

12.3 遅延とパケットロスの模擬

lora_link が、届いたメッセージをどのように遅らせ、捨てるかを、表 12-3 にまとめる。

計算上の遅れは 81.7 ms だが、実測の平均は 85〜86 ms だった。差の約 4 ms は、5 ms ごとにしか確かめないことによる待ち(平均 2.5 ms)と、ROS 2 の受け渡しの時間である。

表 12-3 LoRa のまね

くせ	中身
捨てる	受け取るたびに乱数のくじを引き、3% の確率で捨てる(`loss_rate: 0.03`)。捨てたものは渡さない。送り直し(再送)はしない
遅らせる	捨てなかったものは、「今の時刻 + 送信時間 61.7 ms + 処理の遅れ 20 ms」を渡す時刻としてためておく
渡す	5 ms ごとに(`_deliver`)、渡す時刻を過ぎたものを /rescue/lora/rx に流す。実際に遅れた時間を記録する

12.4 乱数の種

```
self.declare_parameter('seed', -1)        # 負なら実行のたびに変える
seed = self.get_parameter('seed').value
if seed < 0:
    seed = int(time.time() * 1000) % (2 ** 31)   # 実行のたびに違うくじにする
random.seed(seed)
```

くじの出方は、乱数の種で決まる。種が同じなら、毎回同じ順番に同じくじが出る。前は種を 1 に固定していたので、3 回の実行とも同じ 7 回(98 回中)を捨て、届いた割合はどれも 92.9% だった。3% なら捨てるのは平均 2.9 回なので、運の悪いくじを 3 回とも使っていたことになる。

今は、種を**起動した時刻(ミリ秒)**から作るので、実行のたびにくじが変わる。使った種は統計に入れて記録する。同じ結果を再現したいときは、 `params.yaml` の `seed` にその値を書けばよい。

12.5 通信統計

この処理は、 `lora_link_node.py` の 80 行目にある `_publish_stats` で行う。

1 秒ごとに、次の値を /rescue/lora/stats に流す。mission はこれを報告書の `lora` に入れる。

lora_link が 1 秒ごとに流す統計の項目と、1 回目の実行での値を、表 12-4 に示す。

このノードがまねしていないことは、次のとおりである。距離や障害物による届きやすさの変化、3 台に同時に送ったときの電波のぶつかり合い、日本の 920 MHz 帯の送信時間の規則による待ち、再送。どれも実機では確かめる必要がある。

表 12-4 lora_link が流す統計

項目	意味	1 回目の値
n_tx / n_rx	送信数 / 受信数	98 / 94
airtime_s	送信時間の合計	6.046 s
duty	送信時間の割合 = 送信時間の合計 ÷ 起動からの時間	1.74%
delay_ms_mean / max	実際に遅れた時間の平均 / 最大	85.0 / 86.7 ms
seed	使った乱数の種	1388523232

12.6 設計上の考慮点

SF を大きくした場合

SF を 7 から 10 に変えると、1 回の送信時間は 61.7 ms から 371 ms になる。届くまでの遅れは、処理の 20 ms を足して約 391 ms になる。それでも、1 秒ごとの送信には間に合う。

乱数の種を固定した場合

乱数の種を 1 に固定して 3 回実行すると、3 回とも同じくじになり、同じ回数だけ捨てる。送信の回数とタイミングが同じなら、届いた割合も 3 回とも同じになる。

13 章 フォイルボードの制御

ボードは、Gazebo の物理計算で動いているのではない。このノードが 0.05 秒ごとに「次はどこにいるか」を自分で計算し、その位置に Gazebo の上のボードを置き直している(`set_pose_vector`)。ドローンは本物に近い物理計算で飛んでいるが、ボードは速さと曲がり方の上限だけを持つ簡単な計算で動かしている。Gazebo の上のボードは、重力も当たり判定もないモデルである。

13.1 ボードの状態変数

1 台のボードが持つ主な値を、表 13-1 に示す。これらの値は、 `board_node.py` の 42 行目にある `class Board` で決めている。

表 13-1 class Board の主な値

値	意味
`n, e, psi`	ボードの位置(北・東)と向き。向きは北を 0 として時計回りのラジアン
`u, r`	水に対する速さ [m/秒] と、曲がる速さ [rad/秒]
`z`	船体の高さ。0.05 m で浮かんでいて、離水すると 0.32 m になる
`status`	IDLE(砂浜で待機)・TRANSIT(航行中)・APPROACH(接近)・HOLD(止まって待機)
`target`	LoRa で届いた遭難者の推定位置・推定時刻・海流

13.2 LoRa メッセージの受信

この処理は、 `board_node.py` の 84 行目にある `_on_rx` で行う。

- `dispatch` (ボードの発進)は、そのボードが **IDLE のときだけ**受け付ける。状態を TRANSIT にして走り出す。
- `update` (遭難者の位置の更新)は、**今向かっている遭難者のものだけ**受け付け、 `target` を置き換える。

遭難者の位置の更新が届かなかった 1 秒の間も、ボードは前に届いた位置・時刻・海流から予測して進み続ける。

13.3 先回り誘導の計算

この処理は、 `board_node.py` の 99〜111 行目にある `_guidance` で行う。

```
pn = t['north'] + t['vn'] * (now - t['t_est'])     # ① 今の人の位置(予測)
pe = t['east']  + t['ve'] * (now - t['t_est'])
d = math.hypot(pn - b.n, pe - b.e)                 #    ボードからの距離
t_go = d / max(b.u, 2.0)                           # ② 着くまでの時間(速さが 2 m/秒未満なら 2 m/秒とみなす)
pn += t['vn'] * t_go                               # ③ 着くころの人の位置
pe += t['ve'] * t_go
an = pn - s * (pn - b.n) / d                       # ④ そこからボードの側へ 2 m 手前
ae = pe - s * (pe - b.e) / d
```

例として、人が北 −145 m・東 −23 m にいて、ボードが水際(北 −5 m・東 12 m)を出るときを計算する。距離は 144.3 m である。

ボードの速さによって、狙う点がどれだけ先になるかを、表 13-2 に示す。海流は 0.25 m/秒・方位 200° である。

出発直後は、まだ遅いので「着くまでに 72 秒かかる」と見積もり、18 m 先を狙う。速くなるにつれて見積もりが短くなり、狙う点は手前へ戻ってくる。近づくほど「着くまでの時間」は 0 に近づくので、最後は人の今の位置の 2 m 手前を狙うことになる。

表 13-2 ボードの速さと狙う点の位置

ボードの速さ	着くまでの時間	その間に人が流れる距離	狙う点のずれ(北・東)
0〜2 m/秒(出発直後)	72.2 秒	18.0 m	−16.9 m・−6.2 m
4 m/秒	36.1 秒	9.0 m	−8.5 m・−3.1 m
6 m/秒(最高)	24.1 秒	6.0 m	−5.6 m・−2.1 m

13.4 運動の更新

この処理は、`board_node.py` の 113〜153 行目にある `_step` で行う。

ボード 1 台ずつについて、0.05 秒ごとに決めることを、表 13-3 にまとめる。

P 制御の P は Proportional(比例)である。ずれに比例した強さで直すので、ずれが大きいときは速く、小さくなるとゆっくり曲がり、行き過ぎにくい。

止まったあと(HOLD)も、ボードは海流に流され続ける。人も同じ海流で流れるので、2 m の距離は保たれる。これが「浮き具として一緒に流れる」の中身である。

表 13-3 ボード 1 台ずつ決めること

決めること	中身	数字の例
① 曲がる速さ	方位の P 制御で決める。狙う方向と今の向きのずれ × 2.0 の速さで曲がる。上限は 1 rad/秒(57°/秒)。離水中は 6 割(34°/秒)に抑える	ずれ 10° → 20°/秒。ずれ 30° → 60°/秒だが上限の 57°/秒
② 進む速さ	狙う点まで 25 m より遠ければ最高の 6 m/秒。近いほど遅くする。速さは「距離 ÷ 25 × 6」m/秒とし、0.8 m/秒より遅くはしない。向きのずれが 60° を超えたら 4 割に落とす。加速は 1 秒に 1.5 m/秒、減速は 1 秒に 3 m/秒まで	12.5 m なら 3 m/秒。止まっている状態から 6 m/秒までは 4 秒
③ 浮くか	4 m/秒を超えると、船体の高さを 0.32 m へ、1 秒に 0.3 m の速さで上げる(離水)	0.05 m から 0.32 m まで約 0.9 秒
④ 止まるか	狙う点まで 0.8 m 以内なら止まる(HOLD)。速さと曲がる速さを 0 にし、到着の時刻を記録する	人からは約 2 m の所で止まる
⑤ ボードの位置を進める	北へ「速さ × cos(向き) + 海流の北向き」× 0.05 秒、東へ「速さ × sin(向き) + 海流の東向き」× 0.05 秒。砂浜で待っている間(IDLE)は海流に流されない	6 m/秒・南向きなら 0.05 秒で南へ 0.3 m

13.5 Gazebo への表示

この処理は、`board_node.py` の 155〜173 行目にある `_send_poses` で行う。

- 座標を直す。計算は NED(北・東)で行い、Gazebo には ENU(x = 東、y = 北)で渡す。向きも「東から反時計回り」に直す(`yaw_enu = π/2 − psi`)。

- 見た目の傾きをつける。曲がるときは内側に傾け(曲がる速さと進む速さに比例)、離水の途中は少し機首を上げる。計算には使わない、見た目だけの傾きである。
- 発光マーカーを出し入れする。止まるまでは水面の 20 m 下に隠しておき、HOLD になったらボードの上 0.15 m に出す。
- 3 台のボードと 3 個のマーカーの 6 つの位置を、1 回の要求(`set_pose_vector`)でまとめて送る。

0.2 秒ごと(4 回に 1 回)に、3 台のボードの位置・速さ・状態・航跡を `/rescue/boards` に流す(`_publish_state`)。mission はこれで到着を知り、dashboard は地図に航跡を描く。

13.6 設計上の考慮点

出発直後に先を狙う理由

出発直後のボードは、人よりかなり先を狙う。速さが 2 m/秒未満のときは 2 m/秒として、着くまでの時間を長めに見積もるからである。ただし、狙う点は 0.05 秒ごとに計算し直すので、速くなれば狙う点はすぐ手前へ戻る。従って、最初に向く方向が少し下流寄りになるだけで、困ることはない。

狙う点の計算例

たとえば、人まで 120 m あり、ボードが 6 m/秒で走るとする。着くまでの時間は 120 ÷ 6 = 20 秒で、その間に人は 0.25 × 20 = 5 m 流れる。従って、狙う点は人の今の位置から 5 m 先であり、ボードはその点の 2 m 手前を狙う。

2 m 手前で止める理由

ボードは、人の真上ではなく、2 m 手前で止まる。ボードが遭難者にぶつかって、けがをさせないためである。止まったあとは浮き具として、遭難者がつかまれる距離で待つ。

14 章 評価と動画作成

14.1 正解位置の記録

記録は、`scripts/truth_logger.py` が行う。

Gazebo は、動くモデルの位置を `/world/sea_board/dynamic_pose/info` という gz-transport のトピックに流している。truth_logger はこれを聞き、名前が `person_` ・ `board_` ・ `beacon_` で始まるものの位置を、0.5 秒ごとに 1 行ずつ書き出す。

```
{"t": 1792389895.17, "poses": {"person_P1": [−99.952, −3.085], "board_B1": [−5.0, 12.0], ...}}
```

これは 1 回目の出動の約 5 秒後の 1 行である。P1 は北 −100.0 m・東 −3.1 m にいて、B1 はまだ水際で待っている。

- 記録する位置は、Gazebo の ENU から NED に直し、[北, 東] の順で書く。
- 時刻 `t` は UNIX 時刻である。mission の報告書の時刻と同じ時計なので、照らし合わせられる。
- ROS 2 ではなく gz-transport で直接聞くので、AI と制御のノードとはまったくつながっていない。

14.2 遭難者の推定位置と正解位置の比較

評価は、`tests/evaluate_mission.py` が行う。

`tests/evaluate_mission.py` が評価する手順を、表 14-1 に示す。

表 14-1 評価の手順

段階	中身
正解の記録を選ぶ	報告書の出動時刻(`t_start`)より前に書き始めた truth_*.jsonl のうち、一番新しいもの。`--truth` で指定もできる
補間	求めたい時刻の前後 2 行の正解位置を、時刻の割合で結ぶ(`Truth.at`)。0.5 秒の間を直線で補う
T と P の対応	確定した時刻に一番近くにいた P を選び、最後の推定位置と、同じ時刻の P の位置が 5 m 以内なら対応させる。1 人の P には 1 つの T だけ
ボードとの距離	ボードが止まった 1 秒後の、ボードと P の正解位置の距離
事案発生の時刻	出動時刻 − 120 秒。Gazebo の起動待ちの時間は含めない
出力	表を画面に出し、軌跡の地図を報告書と同じ名前の PNG に描く(レポート編 図 4-1)

14.3 録画

録画は、`scripts/record_demo.sh` が行う。

1. ffmpeg で画面全体(2 つのモニタ)を録画し始める。左のモニタに dashboard、右のモニタに Gazebo の 3D 画面を出している。
2. 録画を始めた時刻を `録画.start` に書いておく。動画を切る位置の計算に使う。
3. 2 秒後に AI 一式(`start_ai.sh`)を起動する。
4. 新しい報告書(mission_*.json)ができたら、4 秒待って録画を止める。

起動の順番と時間には、注意が必要である。遭難者は Gazebo を起動した瞬間から流れ始める(レポート編 1.6 節)。`start_sim.sh` から `record_demo.sh` までを約 30 秒以内に続けて行う。録画し直した最初の 3 回は、待ち時間が約 55 秒になり、3 人目が捜索範囲から外れて見つからなかった。

14.4 動画の作成

動画は、`scripts/make_video.sh` が作る。

`make_video.sh` が動画をどう組み立てるかを、表 14-2 に示す。

表 14-2 動画の組み立て

段階	中身
本編を切る位置	報告書の出来事から「全員にボードが到着」の時刻を探し、録画の開始時刻との差 + 8 秒(`TAIL`)を求める。1 回目は録画の 316.5 秒目で切った。帰還と着陸の場面は入れない
画面の合成	左のモニタ(2048×1152)を 1920×1080 に縮め、右のモニタの 3D 画面を切り出して右上に重ねる。右上に「2倍速」の印を重ねる
速さ	映像の時刻を 2 で割って 2 倍速にする(`SPEED=2`)。本編は 316.5 ÷ 2 ≒ 158 秒
前後のカード	前にタイトル(5 秒)、後ろに結果(7 秒)を付ける。文字は `make_cards.py` が Pillow で描き、結果の数字は evaluate_mission.py の出力をそのまま使う
できあがり	5 + 158 + 7 = 170 秒(2 分 50 秒)。課題の条件(1〜3 分)に収まる

14.5 設計上の考慮点

truth_logger が救助に影響しないこと

truth_logger を止めて実行しても、救助の結果は変わらない。truth_logger はどのノードともつながっていないからである。止めると、評価ができなくなるだけである。

再生速度と動画の長さ

本編は、録画の 316.5 秒目までである。たとえば 3 倍速にすると、本編は 316.5 ÷ 3 ≒ 106 秒になり、前後のカード 5 秒と 7 秒を足して、全体は 118 秒になる。

(解説資料はここまで)